Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

June 7, 2026

Generated June 7, 2026

---

1 Home

2 Open-FDD

Open-FDD is an open-source platform for **fault detection**, **BACnet integration**, and **operator analytics** at the building edge. It combines a React dashboard, FastAPI Operator Bridge, Python Rule Lab, local feather historian, and optional Brick/RDF modeling.

Who it is for

Audience	Start here
IT / controls engineer trying the stack	Quick Start — Docker
Developer contributing or customizing	Developer Guide
Integrator wiring APIs	Appendix — API routes
Analyst authoring FDD rules	Rule Cookbook · Fault Codes

2.1 What you get

Component	Purpose
Operator Bridge	Web API + dashboard (trends, faults, Rule Lab, model tools)
BACnet tools	Discover, read, poll, optional supervised writes
Rule Lab	Arrow-native `apply_faults_arrow(table, cfg)` rules on feather historian data
Local historian	Feather-based telemetry store on the edge host
Brick / RDF model	Equipment and point semantics for bindings and analytics
Docker images	Published on GHCR — pull and run without building from source
Python package	`pip install open-fdd` for offline Arrow rule lint/test (`arrow_runtime`)

2.2 Two paths

2.2.1 Try it with Docker (recommended)

Pull published images from `ghcr.io/bbartling/`, configure auth and BACnet env files, start the stack, open the dashboard.

→ Quick Start

2.2.2 Develop locally

Clone the repo, create `workspace/auth.env.local`, build images, run tests, submit a PR.

→ Developer Guide

2.3 Distribution

Channel	Use when
GHCR images `ghcr.io/bbartling/openfdd-*`	Production or trial edge deploy
This repository	Custom builds, BACnet commissioning, Rule Lab development
PyPI `open-fdd`	Arrow runtime + playground (no full UI); optional `[ml]` for graph experiments

2.4 License

MIT — see repository `LICENSE`.

3 Run with Docker images

4 Run with Docker images

Deploy Open-FDD on a Linux edge host using **three published GHCR images**. No git clone required.

4.1 Images

GHCR image	Service	Role
`ghcr.io/bbartling/openfdd-bridge`	`bridge`	Operator API, dashboard, historian ingest
`ghcr.io/bbartling/openfdd-commission`	`commission`	BACnet discover/

GHCR image	Service	Role
		read/write **and poll loop**
`ghcr.io/bbartling/openfdd-mcp-rag`	`mcp-rag`	Doc-search sidecar

Edge hosts pull **`latest`** from GHCR (three images: bridge, commission, mcp-rag). The retired **`openfdd-bacnet-poll`** image is no longer published.

4.2 1. Install and validate Docker

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
newgrp docker   # or log out/in

docker --version
docker compose version
docker ps
docker run --rm hello-world
```

Expect: `active` / `enabled` for docker, `docker` in `groups`, hello-world succeeds.

4.3 2. Bootstrap the site (one script)

Downloads `docker-compose.yml`, creates `workspace/`, generates `auth.env.local`, and sets `BACNET_BIND` from the host LAN NIC (e.g. `ens192` → `10.200.200.185/24:47808`):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
  https://github.com/bbartling/open-fdd/raw/refs/heads/master/scripts/openfdd_edge_bootstrap.sh
bash /tmp/openfdd_edge_bootstrap.sh
```

Bootstrap + pull + start in one go:

```
bash /tmp/openfdd_edge_bootstrap.sh --start
```

Auth — bootstrap writes `~/open-fdd/workspace/auth.env.local` (gitignored). This file holds the API signing secret and one username/password per role. Use it for your first dashboard login:

```
cat ~/open-fdd/workspace/auth.env.local
```

Variable	Role
`OFDD_AUTH_SECRET`	

Variable	Role
	Signs session tokens (do not share)
`OFDD_OPERATOR_*`	Read-only operator
`OFDD_INTEGRATOR_*`	**Default dashboard login** — commissioning + BACnet
`OFDD_AGENT_*`	Agent / automation

Example shape (passwords are **random on first bootstrap only** — existing file is kept on re-runs):

```
OFDD_AUTH_SECRET=<random>
OFDD_OPERATOR_USER=operator
OFDD_OPERATOR_PASSWORD=<random>
OFDD_INTEGRATOR_USER=integrator
OFDD_INTEGRATOR_PASSWORD=<random>
OFDD_AGENT_USER=agent
OFDD_AGENT_PASSWORD=<random>
```

Sign in as **integrator** with the password from that file → First login and health check.

Action	Touches `auth.env.local`?
First `bash … bootstrap.sh --start`	Creates file with random passwords (if missing)
`bash … bootstrap.sh --start` again	**Keeps** your file — does not overwrite
`bash … bootstrap.sh --restart`	**Keeps** your file — only restarts bridge to reload it
`nano auth.env.local` + `--restart`	Uses **your** edited passwords
`--force-auth`	**Regenerates** random passwords (opt-in only)

The **bridge** container reads `auth.env.local` at startup only. After you change passwords (manual edit or `--force-auth`), recreate bridge (env files are not reloaded by `restart` alone):

```
cd ~/open-fdd && docker compose up -d --force-recreate bridge
```

Regenerate passwords and reload (stack already running):

```
bash /tmp/openfdd_edge_bootstrap.sh --force-auth --restart --show-secrets
```

Manual edit then reload:

```
nano ~/open-fdd/workspace/auth.env.local
bash /tmp/openfdd_edge_bootstrap.sh --restart
```

Options:

Flag	Purpose
`--start`	`docker compose pull && up -d` after layout; curls `http://127.0.0.1:8765/health`
`--image-tag TAG`	default `latest`
`--repo-ref BRANCH`	branch for `compose.edge.yml` if not on `master` yet
`--force-auth`	regenerate `auth.env.local` (restarts bridge if stack is up)
`--restart`	recreate bridge to reload `auth.env.local`; curls `/health` after
`--show-secrets`	print passwords at end (lab only)

From a repo checkout: `./scripts/openfdd_edge_bootstrap.sh --start`

The script prints **BACnet NIC**, **BACNET_BIND**, and file paths — **validate** `commission.env` before polling OT devices:

```
nano ~/open-fdd/workspace/bacnet/commissioning/commission.env
```

> If you used `--start`, the stack is already up. Next step → First login and health check.

4.3.1 Manual start (optional)

Use this only if you ran bootstrap **without** `--start` and want to bring the stack up yourself:

```
cd ~/open-fdd
docker compose pull
docker compose up -d
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

Expected: **bridge**, **commission**, **mcp-rag** — all `Up`. Then → First login and health check.

4.4 Long-term operation

4.4.1 Survive power cycles

All services use `restart: unless-stopped`. After reboot:

```
cd ~/open-fdd && docker compose ps
curl -sf http://127.0.0.1:8765/health
```

4.4.2 Start / stop / restart

```
cd ~/open-fdd
docker compose up -d          # idempotent
docker compose stop
docker compose restart commission
docker compose logs -f --tail 100 commission
```

> Never run `docker compose down -v` or delete `workspace/` on a live site.

4.4.3 LAN access

Put **Caddy** (or nginx) on port 80 → `127.0.0.1:8765`, or expose 8765 through the firewall.

4.5 Next steps

→ First login and health check
→ Updating the stack — `./scripts/openfdd_site_update.sh`

5 First login and health check

6 First login and health check

Quick checks after `docker compose up -d` on the edge host. Services use `restart: unless-stopped` — after a reboot, run `docker compose ps` once Docker is up (see Run with Docker).

6.1 Public health

```
curl -s http://127.0.0.1:8765/health
```

Through Caddy on port 80:

```
curl -s http://127.0.0.1/health
```

Expect `"ok": true` and `"auth_required": true` when auth is configured.

6.2 Login

1. Open the dashboard (`http://<host-lan-ip>/` or `:8765`).
2. Sign in with credentials from `workspace/auth.env.local`.

```
curl -s -X POST http://127.0.0.1:8765/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"integrator","password":"<from-auth-env>"}'
```

6.3 Smoke checklist

Check	Pass
`docker compose ps`	bridge, commission, mcp-rag **Up**
`GET /health`	`200`, `ok: true`
Login	Token returned
BACnet poll	`samples.csv` growing (`tail -1 workspace/bacnet/polls/samples.csv`)
Historian	Files under `workspace/data/feather_store/`

6.4 Stack detail (authenticated)

Integrator token required:

```
TOKEN=$(curl -sf -X POST http://127.0.0.1:8765/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"integrator","password":"..."}' | jq -r .token)
curl -sf http://127.0.0.1:8765/health/stack -H "Authorization:
Bearer $TOKEN" | jq .
```

Look for `bacnet_poll` green/yellow in the services list.

6.5 Troubleshooting

Symptom	Fix
401 everywhere	Configure `workspace/auth.env.local`
Empty BACnet discover	Fix `BACNET_BIND` in `commission.env` — network setup
No poll rows	Commission container running? `points.csv` has enabled rows?
LAN browser timeout	Open firewall port 80 or 8765

Advanced probes (model graph, Rule Lab, compose profiles): Developer health check.

7 Quick Start

8 Quick Start

Run Open-FDD on a Linux edge host using **three GHCR Docker images**. No git clone on the host.

Step	Page
1	Run with Docker images — bootstrap script + `docker compose up`
2	First login and health check
3	Updating the stack — backup + pull

One-liner bootstrap (edge host):

```
        curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
          https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_edge_bootstrap.sh
        bash /tmp/openfdd_edge_bootstrap.sh --start
```

Prerequisites: Linux, Docker enabled on boot, BACnet LAN if polling OT equipment.

> **BACnet writes are disabled by default.** See Write safety.

Stack: `bridge` + `commission` (BACnet + poll) + `mcp-rag`. Details: Containers.

9 Updating the stack

10 Updating the stack

Upgrade GHCR images on the edge host: **backup `workspace/`, pull new tags, recreate containers**. No git pull on the host.

10.1 Before you start

1. SSH to the edge host (`cd ~/open-fdd`).
2. Pick a tag from GitHub Packages — default is **`latest`**.
3. Short maintenance window (containers restart briefly; `restart: unless-stopped` brings them back after reboot).

10.2 Fetch helper scripts (no git clone)

If you bootstrapped with `openfdd_edge_bootstrap.sh`, scripts are already under `~/open-fdd/scripts/`. Otherwise:

```
mkdir -p ~/open-fdd/scripts
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_backup.sh \
  https://github.com/bbartling/open-fdd/raw/refs/heads/master/scripts/openfdd_site_backup.sh
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_update.sh \
  https://github.com/bbartling/open-fdd/raw/refs/heads/master/scripts/openfdd_site_update.sh
chmod +x ~/open-fdd/scripts/openfdd_site_*.sh
```

10.3 One-command upgrade (recommended)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Pulls **`:latest`** by default, verifies all three images exist on GHCR, recreates containers, and hits `/health`.

10.4 Step by step

10.4.1 1. Backup site state

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Archives `workspace/` (feather, BACnet CSVs, model, auth) under `~/openfdd-backups/<timestamp>/`.

Custom backup location:

```
BACKUP_ROOT=~/openfdd-backups/manual ./scripts/openfdd_site_backup.sh
```

10.4.2 2. Verify images (optional)

```
export NEW_TAG="${NEW_TAG:-latest}"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-commission:${NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:${NEW_TAG} >/dev/null &&
echo "All images exist for ${NEW_TAG}"
```

10.4.3 3. Pull and recreate

```
cd ~/open-fdd
./scripts/openfdd_site_update.sh
```

Manual equivalent (same as the script):

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

10.4.4 4. Verify

```
docker compose ps
curl -sf http://127.0.0.1:8765/health | jq .
tail -1 workspace/bacnet/polls/samples.csv
ls -lah workspace/data/feather_store/
```

→ Health check

10.5 Dashboard UI updates

If the release changed the React UI, copy a new `workspace/api/static/app/` build onto the host before or after the image pull. Image pull alone may not change the browser bundle hash.

10.6 Rollback

Restore `docker-compose.yml` from backup and pull again, or re-run bootstrap from a known-good `master` commit. GHCR keeps only the **three newest** versions per image; early sites should rely on **`latest`** plus `workspace/` backups.

`workspace/` is untouched by image-only upgrades.

10.7 Safe cleanup

```
docker image prune -f
```

> Never run `docker compose down -v`, `docker volume prune`, or delete `workspace/` on a live site.

10.8 Next steps

→ Run with Docker images — bootstrap a new host
→ Health check

11 Local development

12 Local development

12.1 Clone and auth

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local
# Edit integrator/operator passwords and OFDD_AUTH_SECRET (32+ chars)
```

12.2 Start stack

```
./scripts/build_and_test.sh          # dashboard build + pytest
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

URL	Service
`http://127.0.0.1:8765`	Bridge (direct)
`http://127.0.0.1:8765/docs`	OpenAPI

12.3 BACnet bench overlay (optional)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
# commission.env — set BACNET_BIND to your OT interface
```

12.4 Docs preview

```
cd docs && bundle install && bundle exec jekyll serve
# http://127.0.0.1:4000/open-fdd/
```

13 Build Docker images

14 Build Docker images

14.1 Local build

```
./scripts/docker_build.sh
# Optional tar for air-gap:
./scripts/docker_build.sh --save
```

Images: `openfdd-bridge`, `openfdd-commission`, `openfdd-mcp-rag`.

14.2 Publish to GHCR (maintainers)

GitHub Actions workflow **Publish Docker addons** — manual dispatch; publishes **`:latest`** only and prunes GHCR (retired `openfdd-bacnet-poll` removed; keep 3 versions per image).

Maintainer checklist:

1. Green CI on `master`
2. Run **Publish Docker addons** workflow
3. Verify `ghcr.io/bbartling/openfdd-{bridge,commission,mcp-rag}:latest`
4. Edge hosts: `docker compose pull && docker compose up -d`

Details live in `.github/workflows/` and `scripts/docker_build.sh` — not duplicated here.

15 Developer Guide

16 Developer Guide

Clone, build, test, and contribute to Open-FDD.

Topic	Page
Local setup	Local development
Docker builds	Build Docker images
Health checks	Health checks (developer)
Tests & CI	Tests and CI
PRs	Contributing

AI-assisted contributors: see repository `AGENTS.md` for workspace layout, skill routing, and agent policy (one paragraph in public docs; details stay internal).

17 Tests and CI

18 Tests and CI

18.1 Local tests

```
        ./scripts/build_and_test.sh           # UI + workspace bridge pytest
        python3 -m pytest open_fdd/tests -q   # PyPI package tests
```

```
cd workspace/dashboard && npm test    # dashboard unit tests (if configured)
```

18.2 CI (GitHub Actions)

Workflow	Trigger	Checks
`ci.yml`	PR / push	pytest, bridge security audit, dashboard build, Jekyll docs
`docs-pages.yml`	push `master`	GitHub Pages site
`publish-open-fdd.yml`	tag `open-fdd-v*`	PyPI wheel
`publish-docker-addons.yml`	manual	GHCR images

18.3 Docs build (same as CI)

```
cd docs && bundle exec jekyll build -d /tmp/open-fdd-site
```

Fix any template or link errors before opening a PR that touches `docs/`.

19 Contributing

20 Contributing

20.1 Expectations

- Open an issue or discuss large changes before big refactors.
- Keep PRs focused; include test or docs updates when behavior changes.
- Do not commit secrets (`auth.env.local`, Ansible `secrets/*.env.local`, commissioning CSVs with real site data).
- Follow existing code style in `workspace/api/` and `workspace/dashboard/`.

20.2 Pull request checklist

☐

- [ ] Tests pass locally (`./scripts/build_and_test.sh` or scoped pytest)
- [ ] Docs updated if user-facing behavior changed
- [ ] `docs/` Jekyll build succeeds
- [ ] No credentials in diff

20.3 Repository layout (short)

Path	Contents
`workspace/api/`	Operator Bridge (FastAPI)
`workspace/dashboard/`	React SPA
`open_fdd/`	PyPI package (`arrow_runtime`, `playground`)
`docker/`	Compose files and image contexts
`infra/ansible/`	Edge deploy playbooks
`docs/`	This site (Jekyll + Just the Docs)

20.4 Security issues

Do not file public issues for vulnerabilities. Contact the maintainer or use GitHub private security advisories.

20.5 Agent-assisted development

Cursor/Claude contributors: read `AGENTS.md` and `skills/` for automation boundaries. Public docs intentionally avoid agent playbook detail.

21 Health checks (developer)

22 Health checks (developer)

Extended validation for engineers with a **full git checkout** — CI, compose overlays, and pre-release gates.

IT operators on edge hosts should use Quick Start — health check instead.

22.1 Clone and local stack

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local   # loopback dev only
./scripts/docker_build.sh                                 # or: ./scripts/openfdd_stack.sh prod
./scripts/openfdd_stack.sh up
```

GHCR production pull (no local build):

```
OPENFDD_IMAGE_TAG=2026.06.07-edge ./scripts/openfdd_stack.sh prod
```

22.2 Stack health script

```
./scripts/stack_health_check.sh
OPENFDD_BASE_URL=http://192.168.204.18:8765 ./scripts/stack_health_check.sh
./scripts/stack_health_check.sh --require-ollama
```

Probes: containers up, `/health`, `/api/model/health`, sites, tree, SPARQL graph (with integrator auth when configured).

22.3 Compose overlays

Overlay	Use
`docker/compose.dev.yml`	Local dev bind-mount
`docker/compose.bench.yml`	Host-network BACnet lab
`docker/compose.prod.yml`	GHCR images instead of local build
`docker/compose.edge.yml`	IT edge template (copy to `~/open-fdd/docker-compose.yml`)

```
docker compose -f docker/compose.dev.yml -f docker/compose.bench.yml ps
docker compose -f docker/compose.dev.yml config --images
```

22.4 BACnet poll pipeline

```
tail -1 workspace/bacnet/polls/samples.csv
cat workspace/data/bacnet_ingest_state.json
docker compose logs --since 10m commission | grep -i poll
docker compose logs --since 10m bridge | grep -i ingest
```

22.5 pytest (bridge security / BACnet)

```
        PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
```

22.6 Pentest production stack (LAN ZAP)

```
        ./scripts/pentest_production_stack.sh start
        ./scripts/pentest_production_stack.sh verify
```

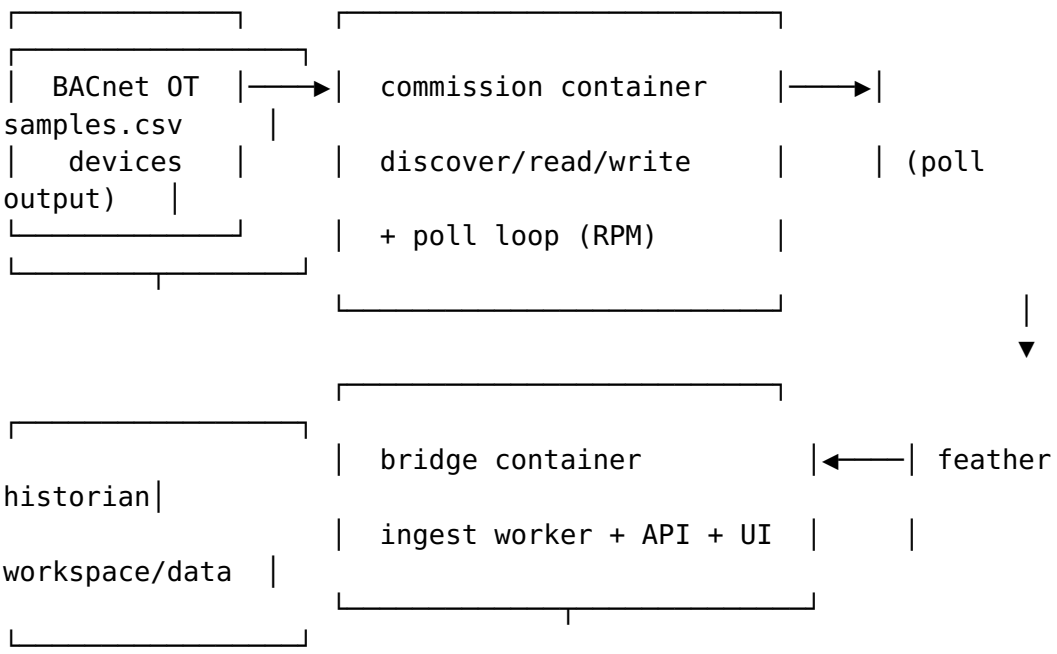
See `workspace/deploy/PENTEST.md`.

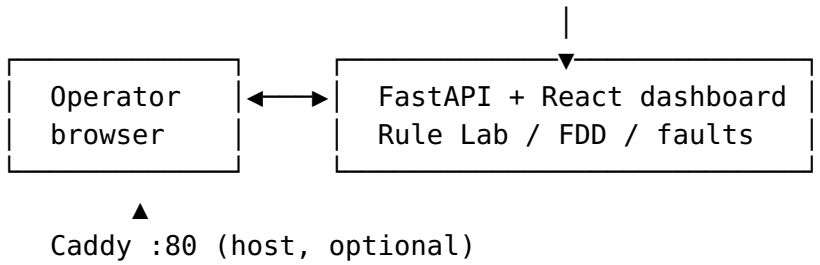
22.7 Post-deploy insurance (inventory hosts)

For automated checks against named inventory hosts, see `infra/ansible/scripts/post_deploy_check.sh` (maintainer tooling — not required for IT docker-pull sites).

23 System overview

24 System overview





24.1 Docker edge stack (production)

Three GHCR images — see Containers:

Container	BACnet OT	Operator LAN
bridge	No (HTTP only)	Yes, via Caddy `:80` or loopback `:8765`
commission	Yes (host network, `:47808`)	No (internal `:8767`)
mcp-rag	No	No (internal `:8090`)

BACnet **polling** is not a separate container — it runs inside **commission**. The bridge only **ingests** poll CSV into feather.

24.2 Components

Layer	Responsibility
Operator Bridge	Auth, REST/WebSocket API, compiled dashboard, Rule Lab, historian ingest
BACnet commission	Who-Is, read/write (gated), **scheduled poll loop** → `samples.csv`
Historian	Feather files per point; local-first retention
FDD runner	`workspace/data/rules_py/*.py` on timer or `POST /api/rules/batch`
Model	Brick TTL + `model.json` bindings (`fdd_input`, equipment tree)
MCP RAG	Optional doc retrieval for agent tools
Caddy	Host reverse proxy `:80` → bridge (typical edge)
Ollama	Optional host LLM for building insight (not required for FDD)

24.3 Deployment

IT operators: Quick Start — Docker (`docker compose pull`, `restart: unless-stopped`, Docker enabled on boot).

24.4 Optional PyPI-only path

`pip install open-fdd` ships Arrow runtime + playground lint helpers **without** the Operator Bridge UI. Use for offline rule tests or CI. See Appendix — Python package.

25 Arrow recipes

26 Arrow recipes (default)

Open-FDD 3.0 Rule Lab rules use **`apply_faults_arrow(table, cfg, context)`**, **`pyarrow.compute`**, and **module constants** (no `config.json`).

26.1 Simple threshold

```
import pyarrow.compute as pc

VALUE_COLUMN = "zone_temp"
MAX_ZONE_TEMP = 78.0


def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table[VALUE_COLUMN], MAX_ZONE_TEMP)
```

26.2 Supply air temp high

```
import pyarrow.compute as pc

VALUE_COLUMN = "sa-t"
HIGH_F = 75.0


def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(pc.cast(table[VALUE_COLUMN], "float64"),
HIGH_F)
```

26.3 Fan commanded but no airflow

```
        import pyarrow.compute as pc

        FAN_CMD = "fan-cmd"
        AIRFLOW = "supply-cfm"
        FAN_ON = 0.5
        MIN_CFM = 500.0


        def apply_faults_arrow(table, cfg, context=None):
            fan_on = pc.greater(table[FAN_CMD], FAN_ON)
            low_airflow = pc.less(table[AIRFLOW], MIN_CFM)
            return pc.and_(fan_on, low_airflow)
```

26.4 Flatline (1 h rolling window)

```
        import pyarrow.compute as pc
        from open_fdd.arrow_runtime.windows import arrow_rolling_max,
arrow_rolling_min

        VALUE_COLUMN = "oa-t"
        WINDOW_SAMPLES = 12
        FLATLINE_TOLERANCE = 0.1


        def apply_faults_arrow(table, cfg, context=None):
            vals = pc.cast(table[VALUE_COLUMN], "float64")
            spread = pc.subtract(arrow_rolling_max(vals, WINDOW_SAMPLES),
arrow_rolling_min(vals, WINDOW_SAMPLES))
            return pc.less_equal(pc.abs(spread), FLATLINE_TOLERANCE)
```

26.5 Sensor out of range

```
        import pyarrow.compute as pc

        VALUE_COLUMN = "oa-t"
        MIN_VALUE = -40.0
        MAX_VALUE = 130.0


        def apply_faults_arrow(table, cfg, context=None):
            vals = pc.cast(table[VALUE_COLUMN], "float64")
            return pc.or_(pc.less(vals, MIN_VALUE), pc.greater(vals,
MAX_VALUE))
```

More templates ship in `open_fdd.playground.arrow_templates` and via `GET /api/playground/arrow-templates`.

Bench examples: `workspace/data/rules_py/bench_*.py`.

Dev kit zip layout: Rule Lab — dev kit zip.

27 Data flow

28 Data flow

28.1 Telemetry ingest

Open-FDD supports three commissioning drivers. Each appends long-format samples to workspace CSVs and writes **feather shards** tagged by `source` (`bacnet`, `modbus`, `json_api`).

28.1.1 BACnet

1. **Commission poll loop** (inside the `commission` container) reads enabled BACnet points from `points.csv` on a schedule (host network, BACnet `:47808`).
2. RPM results append to `workspace/bacnet/polls/samples.csv`.
3. **Bridge ingest worker** loads new rows into `feather_store/bacnet/`.

```
BACnet devices → commission (poll loop) → samples.csv → bridge (ingest) → feather_store/bacnet
```

28.1.2 Modbus TCP

1. Operator configures registers on the **Modbus** tab (or `read_and_store` API).
2. Poll worker reads holding/input registers via Modbus TCP.
3. Samples append to `workspace/modbus/polls/samples.csv` → `feather_store/modbus/`.

28.1.3 JSON API (HTTP/HTTPS)

1. Operator configures REST endpoints on the **JSON API** tab — GET/POST, Bearer or Basic auth, `${ENV:VAR}` placeholders from `json_api.env.local`, optional TLS verify off for self-signed OT gateways.
2. Poll worker issues HTTP requests (one GET can fan out to multiple JSON paths, e.g. OpenWeather `web-oat-t` / `web-rh` / `web-weather-desc`).

3. Samples append to `workspace/json_api/polls/samples.csv` → `feather_store/json_api/`.
4. FDD batch **merges** `json_api` columns with BACnet/Modbus on the same site (nearest timestamp, 30 min tolerance) for cross-source rules.

```
OT REST / weather API → bridge JSON API worker → samples.csv →
feather_store/json_api
                                                    ↘ merged
into FDD frame with bacnet/modbus
```

Showcase: OpenWeatherMap bundle. Details: Driver framework, BACnet polling, JSON API driver, Containers.

28.2 Rule evaluation

1. Rules are Python files in `workspace/data/rules_py/` with metadata in `rules_store.json`.
2. **Bindings** map rules to points via the Brick model (`fdd_input` tags).
3. Batch runner (`POST /api/rules/batch` or host timer) produces `fdd_results.json`.
4. Dashboard **faults** view and check-engine status read aggregated results.

28.3 Model sync

- Commissioning imports update `model.json` / TTL graph.
- `POST /api/model/sync-ttl` refreshes SPARQL-backed tree used by the UI.

28.4 Retention

Feather files grow on disk; retention is configurable (`workspace/data.env.local`). Image upgrades must **preserve** `workspace/data/` — backup before `docker compose up -d --force-recreate`. See Updating the stack.

29 Python recipes (Arrow)

30 Python recipes (Arrow)

All Rule Lab Python rules use **`apply_faults_arrow(table, cfg, context)`** on PyArrow tables with **`pyarrow.compute`** and **module constants**.

See **Arrow recipes** for the canonical cookbook (flatline, spread, OOB, schedule faults).

Shared helpers live in `open_fdd.arrow_runtime.cookbook` and `open_fdd.arrow_runtime.windows`. For console validation, copy **`_kit_lookback_stats`** from Lookback window helper.

```
        from open_fdd.arrow_runtime.windows import arrow_rolling_min,
arrow_rolling_max
        import pyarrow.compute as pc

        VALUE_COLUMN = "oa-t"
        WINDOW_SAMPLES = 12
        FLATLINE_TOLERANCE = 0.1

        def apply_faults_arrow(table, cfg, context=None):
            vals = pc.cast(table[VALUE_COLUMN], "float64")
            spread = pc.subtract(arrow_rolling_max(vals, WINDOW_SAMPLES),
arrow_rolling_min(vals, WINDOW_SAMPLES))
            return pc.less_equal(pc.abs(spread), FLATLINE_TOLERANCE)
```

31 Containers

32 Containers

Open-FDD v3 edge sites run **three** GHCR containers plus optional **host** services (Caddy, Ollama). There is no separate BACnet poll container — polling is part of **commission**.

32.1 GHCR images

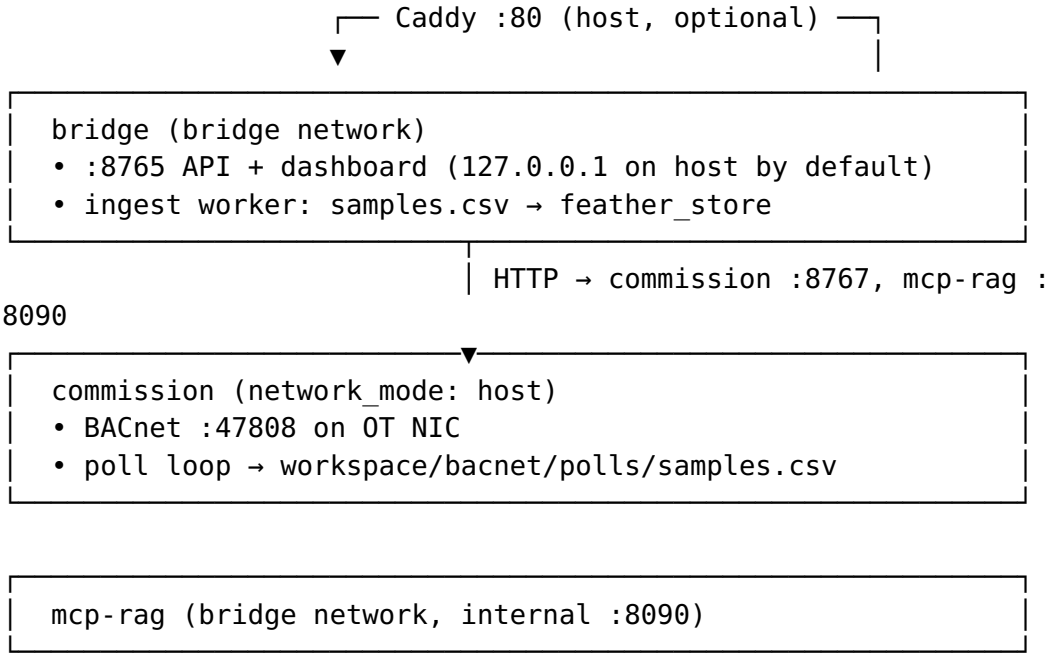
Image	Compose service	Role
	`bridge`	

Image	Compose service	Role
`ghcr.io/bbartling/openfdd-bridge`		Operator API, React dashboard, feather **ingest**
`ghcr.io/bbartling/openfdd-commission`	`commission`	BACnet discover / read / write + **poll loop**
`ghcr.io/bbartling/openfdd-mcp-rag`	`mcp-rag`	Doc-search sidecar for agent tools

Tag: **`latest`** on GitHub Packages. Retired: `openfdd-bacnet-poll` (poll runs inside **commission**). GHCR retains only the **three newest** versions per image.

Template: `docker/compose.edge.yml` → copy to `~/open-fdd/docker-compose.yml`.

32.2 Stack diagram



32.3 Responsibilities

Concern	Container
BACnet Who-Is / read / write	**commission**
BACnet scheduled RPM reads	**commission** (`bacnet_poll_loop`)
CSV → feather historian	**bridge** (`bacnet_poll_worker` thread)
Operator login, Rule Lab, faults UI	**bridge**
Agent doc search	**mcp-rag**

The bridge thread named `bacnet_poll_worker` is **ingest only** — it does not bind BACnet.

32.4 Network and ports (typical edge)

Endpoint	Reachable from	Notes
Caddy `:80`	Building LAN	Reverse proxy to bridge
Bridge `:8765`	Loopback (default)	Bind `127.0.0.1` in compose
Commission : `8767`	Bridge / loopback	HTTP API for BACnet ops
Commission : `47808`	OT BACnet LAN	UDP; requires `network_mode: host`
MCP `:8090`	Bridge container network	Not exposed to LAN by default

32.5 Long-lived deployment

All compose services set `restart: unless-stopped`. After host reboot:

```
sudo systemctl enable docker
cd ~/open-fdd && docker compose ps
```

If containers are down: `docker compose up -d` (idempotent).

See Run with Docker images.

32.6 Persistent data

State lives on the **host** under `workspace/` (bind mount). Containers are replaceable; **backup `workspace/`** before image upgrades.

Path	Contents
`workspace/data/feather_store/`	Historian
`workspace/data/*.json`	Model, rules, FDD results
`workspace/bacnet/commissioning/`	`commission.env`, `points.csv`
`workspace/bacnet/polls/samples.csv`	Poll output
`workspace/auth.env.local`	Login secrets
`workspace/api/static/app/`	Dashboard bundle (if updated separately)

32.7 Optional host services

Service	Role
Caddy	TLS or plain HTTP `:80` → `127.0.0.1:8765`
Ollama	Optional LLM on `:11434` (not in the three-image stack)
FDD loop timer	Optional `openfdd-fdd-loop.timer` on host (`docker exec` batch)

33 Lookback window helper

34 Lookback window helper

Rules that depend on **time history** (flatline, spread, streaks, trim/respond validation) should print a clear window summary in the Rule Lab console or local `run_test.py` output.

34.1 Recommended pattern (copy into `rule.py`)

Use module constants for the lookback you intend; the bridge passes the full feather window into `apply_faults_arrow` based on batch `lookback_hours` (default **1 h** on the scheduled loop; **24 h** when using **Update all records** with chunks).

```
        import pyarrow.compute as pc

        LOOKBACK_HOURS = 3  # tune: 1, 3, 6, 12, 24


        def _kit_lookback_stats(table, *, hours=None):
            """Dev helper — prints row count, timestamp span, and
configured lookback."""
            h = hours if hours is not None else LOOKBACK_HOURS
            if "timestamp" not in table.column_names:
                print(f"lookback={h}h rows={table.num_rows} (no timestamp
column)")
                return
            ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
            tmin = pc.min(ts).as_py()
            tmax = pc.max(ts).as_py()
            if tmin is None or tmax is None:
                print(f"lookback={h}h rows={table.num_rows} start=None
stop=None span=0.00h")
                return
            span_h = (tmax - tmin).total_seconds() / 3600.0
            print(
                f"lookback={h}h rows={table.num_rows} "
                f"start={tmin.isoformat()} stop={tmax.isoformat()}
span={span_h:.2f}h"
            )


        def _kit_value_stats(table, column):
            vals = pc.cast(table[column], "float64")
            print(
                f"column={column} min={pc.min(vals).as_py():.2f} "
                f"max={pc.max(vals).as_py():.2f}
mean={pc.mean(vals).as_py():.2f}"
            )


        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table)
            _kit_value_stats(table, "oa-t")
            # ... fault logic ...
```

34.2 Why not hide this behind a library call yet?

A future `open_fdd.arrow_runtime.kit.lookback_stats(table, hours=3)` helper is reasonable once the Algorithms tab shares the same kit export path. For now, keeping the helper **inline in `rule.py`** matches the constants-first Rule Lab style and prints the same fields in:

- Local `run_test.py`
- Rule Lab **Quick test** console
- Batch run logs

34.3 Lookback vs rolling samples

Concept	Controlled by	Typical use
Historian lookback	Batch `lookback_hours`, FDD loop env	How many rows are in `table`
Rolling window	`WINDOW_SAMPLES` constant (~12 ≈ 1 h @ 5 min poll)	Flatline, spread, consecutive-true

Always call `_kit_lookback_stats(table)` first so operators can confirm the table span matches expectations before interpreting rolling-window faults.

34.4 Batch intervals

Trigger	Default lookback
`openfdd-fdd-loop` / `OFDD_FDD_INTERVAL_MINUTES`	1 h
Rule Lab **Update all records**	24 h (`chunk_hours: 6`, `use_chunks: true`)
Rule Lab Quick test	3 h (UI default)

See Windowing & debugging for `arrow_rolling_min` / `arrow_consecutive_true`.

35 Architecture

36 Architecture

High-level view of the Open-FDD v3 edge stack: **three Docker containers** (`bridge`, `commission`, `mcp-rag`), BACnet polling inside **commission**, historian ingest in **bridge**.

Page	Topic
System overview	Components, diagram, deployment pointer
Data flow	BACnet / Modbus / JSON API → feather → FDD → dashboard
Driver framework	Shared commissioning pattern for all OT sources
Containers	Images, networks, ports, persistence, reboot

Operators: deploy and upgrade via Quick Start — no git clone on the edge host.

BACnet polling: Polling — commission loop only (no separate poll container).

37 Rule Cookbook

38 Rule Cookbook

Practical patterns for **Arrow-native Rule Lab** (`apply_faults_arrow`) on feather historian PyArrow tables.

Page	Use when
Arrow recipes	**Default** — thresholds, flatline, spread, OOB, fan/schedule faults
Python recipes	Same Arrow patterns with shared `open_fdd.arrow_runtime.cookbook` imports
Lookback window helper	`_kit_lookback_stats` — print start/stop timestamps and span
Windowing & debugging	Rolling windows, batch runtime, Rule Lab console

Page	Use when
GL36 algorithm stubs	Doc-only supervisory patterns (trim & respond, plant resets)

All expression rules are **PyArrow columnar** — no YAML rule files, no pandas DataFrames in Rule Lab.

Future graph ML (sklearn / PyG) runs in a separate training service — see GitHub issue #211.

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

39 Windowing & debugging

40 Windowing & debugging

40.1 Lookback vs rolling window

	Historian lookback	Rolling window
Set by	Batch `lookback_hours`, FDD loop, Quick test UI	`WINDOW_SAMPLES` constant in `rule.py`
Typical	1 h scheduled; 24 h Update all	12 samples ≈ 1 h @ 5 min poll
Validate with	`_kit_lookback_stats(table)` — see Lookback window helper	Rule logic + console `flagged` count

40.2 Rolling windows (Arrow)

Use `open_fdd.arrow_runtime.windows` for sample-based rolling min/max and consecutive-true streaks:

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min, arrow_rolling_max, arrow_consecutive_true
```

Cookbook masks (`flatline_1h_mask`, `spread_1h_mask`, `oob_mask`) default to ~12 samples (~1 h at 5 min poll).

Bench rules use constants: `WINDOW_SAMPLES`, `FLATLINE_TOLERANCE`, `OAT_LOW` / `OAT_HIGH`, etc. Legacy `cfg` keys still work for uploaded rules that read `cfg.get(...)`.

40.3 Rule Lab console

Quick-test and batch responses include `backend: arrow`, `ms`, and flagged row counts. Arrow summary events appear in the event console on fault hits.

41 GL36 algorithm stubs

42 GL36 algorithm stubs (doc-only)

Draft **supervisory** PyArrow patterns for the future Algorithms tab. These mirror Niagara GL36 blocks documented in:

README_TRIM_RESPOND.md

FDD rules return **fault masks**. Algorithms return **setpoints or request integers** — the stubs below use `apply_algorithm_arrow` as the planned entrypoint. For early testing in Rule Lab, you can adapt the logic into advisory FDD rules (boolean “sequence unhealthy” flags).

42.1 Shared lookback helper

```
        import pyarrow.compute as pc

        LOOKBACK_HOURS = 6


        def _kit_lookback_stats(table, *, hours=None):
            h = hours if hours is not None else LOOKBACK_HOURS
            ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
            tmin = pc.min(ts).as_py()
            tmax = pc.max(ts).as_py()
            span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
            print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")
```

42.2 VAV zone cooling requests (0–3)

Simplified from GL36 VAV box request generator — zone temp vs cooling setpoint bands.

```
        """GL36 VAV cooling requests — doc stub."""

        import pyarrow.compute as pc

        ZONE_TEMP = "zn-t"
        ZONE_SP = "zn-t-sp"
        ZONE_DEMAND = "zn-cool-loop"
        HIGH_DIFF_F = 5.0
        MED_DIFF_F = 3.0
        LOOP_ON = 95.0
        LOOP_OFF = 85.0


        def apply_algorithm_arrow(table, cfg, context=None):
            _kit_lookback_stats(table)
            tz = pc.cast(table[ZONE_TEMP], "float64")
            sp = pc.cast(table[ZONE_SP], "float64")
            diff = pc.subtract(tz, sp)
            demand = pc.cast(table[ZONE_DEMAND], "float64")
            # Last row request level for kit demo (full port needs timer
state)
            last_diff = diff[-1].as_py() if table.num_rows else 0.0
            last_demand = demand[-1].as_py() if table.num_rows else 0.0
            if last_diff >= HIGH_DIFF_F:
                req = 3
            elif last_diff >= MED_DIFF_F:
                req = 2
            elif last_demand > LOOP_ON:
                req = 1
            else:
                req = 0
            print(f"cooling_requests={req} dT={last_diff:.1f}F
loop={last_demand:.0f}%")
            return {"cooling_requests": req}
```

42.3 AHU duct static trim & respond (advisory fault)

Detect **duct static stuck high** while VAV pressure requests are low — trim & respond may not be trimming.

```
        """AHU duct static trim advisory — doc stub."""

        import pyarrow.compute as pc
```

```
        DUCT_SP = "duct-static"
        DUCT_SP_MAX = 1.50
        DUCT_SP_TRIM_TARGET = 0.80
        VAV_PRESS_REQ_SUM = "vav-press-req-sum"
        LOOKBACK_HOURS = 12


        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
            sp = pc.cast(table[DUCT_SP], "float64")
            reqs = pc.cast(table[VAV_PRESS_REQ_SUM], "float64")
            stuck_high = pc.and_(pc.greater(sp, DUCT_SP_TRIM_TARGET),
pc.less(reqs, 1.0))
            return pc.and_(stuck_high, pc.greater(sp, DUCT_SP_MAX * 0.9))
```

42.4 Chiller plant enable (valve request counting)

From GL36 §5.18.15.2 — count AHUs with CHW valve above threshold.

```
        """Chiller plant enable advisory — doc stub."""

        import pyarrow.compute as pc

        AHU_VALVE_COLS = ["ahu1-clg-vlv", "ahu2-clg-vlv", "ahu3-clg-vlv"]
        REQ_THRESH = 95.0
        DIS_THRESH = 10.0
        MIN_AHUS_REQ = 2


        def apply_algorithm_arrow(table, cfg, context=None):
            _kit_lookback_stats(table)
            requesting = 0
            for col in AHU_VALVE_COLS:
                if col not in table.column_names:
                    continue
                v = pc.cast(table[col], "float64")[-1].as_py()
                if v >= REQ_THRESH:
                    requesting += 1
            enable = requesting >= MIN_AHUS_REQ
            print(f"chiller_enable={enable} requesting_ahus={requesting}/
{MIN_AHUS_REQ}")
            return {"chiller_enable": enable, "requesting_ahus":
requesting}
```

42.5 Hot water plant HWST trim & respond (advisory)

Flags **HWST trimmed too low** while heating requests remain high.

```
        """HWST trim advisory — doc stub."""

        import pyarrow.compute as pc

        HWST = "hw-supply-t"
        HWST_SP = "hw-supply-t-sp"
        HEAT_REQ_SUM = "hw-reset-req-sum"
        LOW_HWST_F = 120.0
        LOOKBACK_HOURS = 6


        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
            temp = pc.cast(table[HWST], "float64")
            reqs = pc.cast(table[HEAT_REQ_SUM], "float64")
            return pc.and_(pc.less(temp, LOW_HWST_F), pc.greater(reqs,
2.0))
```

42.6 Chilled water plant (DP + CHWST reset advisory)

Flags **CHWST at design cold** with low cooling requests — possible stuck 100% plant loop.

```
        """CHW plant reset advisory — doc stub."""

        import pyarrow.compute as pc

        CHWST = "chw-supply-t"
        CHWST_DESIGN_COLD_F = 44.0
        COOL_REQ_SUM = "chw-reset-req-sum"
        LOOKBACK_HOURS = 6


        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
            temp = pc.cast(table[CHWST], "float64")
            reqs = pc.cast(table[COOL_REQ_SUM], "float64")
            return pc.and_(pc.less(temp, CHWST_DESIGN_COLD_F + 1.0),
pc.less(reqs, 1.0))
```

42.7 Testing later

1. Bind historian columns in **Model & assignments** (`fdd_input` / brick types).
2. Copy a stub into Rule Lab as an FDD advisory rule, or wait for the **Algorithms** tab kit export.
3. Run `python run_test.py` from a downloaded kit and confirm `_kit_lookback_stats` prints expected `start` / `stop` / `span`.

Reference implementation (Niagara Java): README_TRIM_RESPOND.md.

43 Dashboard overview

44 Dashboard overview

44.1 Primary areas

Area	Purpose
Home / Building insight	Check-engine status (green/yellow/red), active fault summary
Trends	Point history from feather store
Faults	Rule hits grouped by equipment family
Rule Lab	Edit, lint, and test Python FDD rules (PyArrow kit zip)
Algorithms	Supervisory GL36 trim & respond (coming soon)
Model & assignments	Equipment tree, FDD rule pins, commissioning JSON
BACnet	Discovery, reads, commission workflows
Settings / health	Stack status, auth mode

44.2 Live updates

Dashboard tiles use REST polling and `WS /ws/dashboard` for selected live views (ticket auth via `POST /api/auth/ws-ticket`).

44.3 Roles

Read-only operators see trends and faults; integrators access Rule Lab and model import. See Auth and roles.

45 Auth and roles

46 Auth and roles

46.1 Production (LAN / edge)

- Set `OFDD_AUTH_SECRET` and role passwords in `workspace/auth.env.local`.
- Bridge on non-loopback addresses **requires** auth unless explicit insecure dev flags are set (lab only).

Role	Typical access
`viewer`	Read trends, faults, model tree
`operator`	Run batch FDD, acknowledge views
`integrator`	Rule Lab, bindings, model import
`commission`	BACnet writes (plus write guard)
`agent`	Agent tools / app-edit gates
`admin`	Full configuration

Login: `POST /api/auth/login` → Bearer JWT on API calls.

46.2 Localhost dev

`OFDD_BRIDGE_HOST=127.0.0.1` with `OFDD_AUTH_DISABLED=1` is acceptable on a single machine. **Never** use auth-disabled mode on a building LAN.

Full hardening: Security.

47 Rule Lab

48 Rule Lab

Rule Lab authors **Arrow-native** Python FDD rules against the feather historian. Bench rules use **module constants** at the top of `rule.py` — no browser config panel, no `config.json` in the dev kit zip.

```
        """Bench OA-T out of bounds (Arrow)."""

        import pyarrow.compute as pc

        VALUE_COLUMN = "oa-t"
        OAT_LOW = 68.0
        OAT_HIGH = 88.0
        LOOKBACK_HOURS = 1


        def _kit_lookback_stats(table, *, hours=None):
            h = hours if hours is not None else LOOKBACK_HOURS
            ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
            tmin = pc.min(ts).as_py()
            tmax = pc.max(ts).as_py()
            span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
            print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")


        def _kit_value_stats(table):
            vals = pc.cast(table[VALUE_COLUMN], "float64")
            print(
                f"column={VALUE_COLUMN} min={pc.min(vals).as_py():.2f} "
                f"max={pc.max(vals).as_py():.2f}
mean={pc.mean(vals).as_py():.2f}"
            )


        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table)
            _kit_value_stats(table)
            vals = pc.cast(table[VALUE_COLUMN], "float64")
            return pc.or_(pc.less(vals, OAT_LOW), pc.greater(vals,
OAT_HIGH))
```

48.1 Workflow

1. **Download kit** — PyArrow zip (see below)
2. **Edit constants** locally (`VALUE_COLUMN`, limits, `LOOKBACK_HOURS`, `WINDOW_SAMPLES`)
3. **Run** `pip install -r requirements.txt` then `python run_test.py`
4. **Upload** `rule.py` on Rule Lab (integrator)

Upload validation (Phase B): AST parse, forbidden imports, `apply_faults_arrow(table, cfg, context=None)` signature.

The bridge runs rules on **PyArrow tables** via `open_fdd.arrow_runtime`, and persists `.py` sources under `workspace/data/rules_py/`.

- **Quick test** — `POST /api/playground/test-rule` (default 3 h lookback)
- **Batch** — `POST /api/rules/batch` / `openfdd-fdd-loop` timer (**1 h** default lookback)
- **Update all records** — 24 h lookback, 6 h chunks (`use_chunks: true`)
- **Templates** — `GET /api/playground/arrow-templates`

Pin points to rules via **Model & assignments** (`/model`) commissioning JSON or BACnet tree right-click.

Bench seed: `python scripts/setup_bench_afdd.py`

48.2 Dev kit zip (download)

Download kit on Rule Lab exports `openfdd-rule-kit-<rule_id>.zip` with:

File	Purpose
`rule.py`	Arrow rule + **constants** at top; optional `_kit_lookback_stats` / `_kit_value_stats` helpers
`data.py`	`SITE_ID`, `LOOKBACK_HOURS`, `load_table()` reading `sample.feather`
`sample.feather`	Full historian window for the kit lookback (all rows, not a CSV preview)
`run_test.py`	Runs `rule.apply_faults_arrow(table, {}, context=...)` and prints `flagged=N`
`requirements.txt`	`open-fdd>=3.0.1` and `pyarrow`
`README.md`	Install and run instructions
`column_map.json`	BRICK logical keys → feather column names

File	Purpose
`kit_meta.json`	Export metadata (site, columns, row count, lookback hours)

Not included: `config.json` (retired — tune thresholds as Python constants).

Local test:

```
unzip openfdd-rule-kit-*.zip
cd openfdd-rule-kit-*
pip install -r requirements.txt
python run_test.py
```

Expected console output:

```
site=demo lookback_h=3 rows=…
lookback=3h rows=… start=… stop=… span=2.98h
column=oa-t min=… max=… mean=…
flagged=…
```

48.3 Lookback windows

Rules that need multi-hour history should call `_kit_lookback_stats(table)` (see Lookback window helper). Pass `hours=1`, `3`, `6`, `12`, or `24` to validate timestamp span in the console.

Rolling-window rules (flatline, spread) use `WINDOW_SAMPLES` (~12 samples ≈ 1 h at 5 min poll) — see Windowing & debugging.

48.4 Algorithms (supervisory — coming soon)

GL36 trim & respond and plant reset sequences will live on the **Algorithms** tab with the same zip kit shape but `apply_algorithm_arrow` outputs. Reference: README_TRIM_RESPOND.

48.5 Related

- Model workflow
- Arrow recipes
- GL36 algorithm stubs (doc-only)

49 JSON API driver

50 JSON API driver

Poll **HTTP or HTTPS** REST endpoints on the OT LAN (or public APIs such as OpenWeatherMap) and store extracted JSON values in the feather historian (`source=json_api`).

Use this driver for gateways, micro-PLCs, IoT hubs, vendor APIs, and **web weather** when you want to cross-check shaky local outdoor-air sensors against a reference.

50.1 Supported features

Feature	Support
Methods	`GET`, `POST`
Transport	`http://` and `https://`
TLS certificate verify	On by default; disable for self-signed OT gateways
Authentication	None, **Bearer token**, **HTTP Basic**, or **query-string API keys via env**
Env placeholders	`${ENV:VAR}` or `${VAR}` in URL, headers, body, bearer token
Custom headers	Optional `headers` map (merged with auth headers)
Value extraction	Dot-path into JSON body (`title`, `main.temp`, `weather.0.description`)
Multi-extract poll	One HTTP request per URL group; fan-out to multiple `json_path` rows
Polling	1 / 5 / 10 / 15 / **20** minutes (same worker pattern as BACnet/Modbus)
Historian	`workspace/json_api/polls/samples.csv` → `feather_store/json_api/`
FDD merge	Scheduled FDD merges `bacnet` + `modbus` + `json_api` columns by nearest timestamp

50.2 Secrets: `json_api.env.local`

Copy `workspace/json_api.env.example` → `workspace/json_api.env.local` (gitignored). The bridge loads this file at startup and when `run_local.sh` starts the stack.

Use placeholders in commissioning URLs so API keys never land in git:

```
https://api.openweathermap.org/data/2.5/weather?q=${ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=${ENV:OPENWEATHER_UNITS}
```

Check which vars are configured: `GET /api/json-api/env/status`.

50.3 OpenWeatherMap showcase (recommended demo)

This is the flagship example of what the generic JSON API driver can do — **one URL, three historian columns**, env-based API key, browser tree like BACnet/Modbus.

50.3.1 1. Configure env

```
cp workspace/json_api.env.example workspace/json_api.env.local
# Edit: OPENWEATHER_API_KEY, OPENWEATHER_CITY, OPENWEATHER_UNITS (imperial = °F)
```

Restart the bridge (or `./scripts/run_local.sh restart`) so env vars load.

50.3.2 2. Register from the UI

1. Sign in as **integrator** → **JSON API** tab.
2. Open the **OpenWeatherMap showcase** panel.
3. Click **Register OpenWeather bundle (3 points, poll 20 min)**.

Three endpoints appear under `api.openweathermap.org` in the commissioning tree:

Label	JSON path	Historian column	Units
web-oat-t	`main.temp`	`web-oat-t`	degF / degC
web-rh	`main.humidity`	`web-rh`	%
	`weather.0.description`	`web-weather-desc`	text

Label	JSON path	Historian column	Units
web-weather-desc			

Poll worker deduplicates: **one GET** serves all three extractions per cycle.

50.3.3 3. Headless / script

```
# Standalone fetch (like the original playground tester)
python3 scripts/openweather_json_api_example.py

# Register via REST (integrator login)
python3 scripts/openweather_json_api_example.py --register --base
http://127.0.0.1:8000
```

50.3.4 4. FDD: local OA-T vs web weather

With BACnet `oa-t` and JSON API `web-oat-t` in the same site, scheduled FDD merges both sources. Example rule:

`workspace/data/rules_py/oat_vs_web_spread_1h.py` — flags when `|oa-t − web-oat-t| > 8 °F` (stuck or drifting outdoor-air sensor).

Enable it in Rule Lab, bind to your OA-T BACnet point, and run **Update all records**.

50.4 Commissioning (UI)

1. Sign in → **JSON API** tab (below Modbus).
2. Enter URL, method, JSON path, and label (historian column name).
3. Set **Auth** if the device requires it:
   - **Bearer token** — common for API keys and JWT gateways (or use `${ENV:TOKEN}` in the field).
   - **HTTP Basic** — username/password for legacy OT web servers.
4. **TLS verify** — leave checked for public HTTPS; uncheck for `https://192.168.x.x` with a self-signed certificate.
5. Click **Request & store to historian** — endpoint is saved to `endpoints.csv` and a feather shard is written.
6. In the tree, right-click or bulk-select → **Poll 1 min** (etc.).

50.4.1 Bench preset

Use the **Todo title** preset against JSONPlaceholder to verify outbound HTTPS without field hardware.

50.5 Authentication examples

50.5.1 Query-string API key via env (OpenWeather)

```
        {
          "url": "https://api.openweathermap.org/data/2.5/weather?q=$
{ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=$
{ENV:OPENWEATHER_UNITS}",
          "method": "GET",
          "json_path": "main.temp",
          "label": "web-oat-t",
          "auth_type": "none"
        }
```

50.5.2 Bearer token (API key)

```
        {
          "url": "https://192.168.10.50/api/v1/status",
          "method": "GET",
          "json_path": "sensors.zone_temp",
          "label": "zone-temp",
          "auth_type": "bearer",
          "bearer_token": "${ENV:GATEWAY_API_KEY}",
          "verify_tls": false
        }
```

50.5.3 HTTP Basic

```
        {
          "url": "http://192.168.10.50/data.json",
          "method": "GET",
          "json_path": "value",
          "label": "sensor-value",
          "auth_type": "basic",
          "basic_user": "operator",
          "basic_password": "changeme"
        }
```

50.5.4 POST with JSON body

```
        {
          "url": "https://gateway.local/api/read",
          "method": "POST",
```

```
      "json_path": "result.pv",
      "label": "pv",
      "body": { "point": "AHU-1_SAT", "command": "read" },
      "auth_type": "bearer",
      "bearer_token": "…"
    }
```

Credentials in `endpoints.csv` are redacted in API responses (***). Prefer `${ENV:…}` for production keys.

50.6 REST API

Method	Path	Role	Description
GET	`/api/json-api/env/status`	operator+	Which env vars are configured (no secret values)
POST	`/api/json-api/presets/openweather`	integrator+	Register 3-point OpenWeather bundle + optional poll
GET	`/api/json-api/driver/tree`	operator+	Commissioning tree grouped by host
GET	`/api/json-api/poll/status`	operator+	Poll worker status
POST	`/api/json-api/poll/once`	integrator+	Run one poll cycle
POST	`/api/json-api/request`	operator+	One-shot HTTP request (no store)
POST	`/api/json-api/read_and_store`	integrator+	Request + CSV + feather ingest
POST	`/api/json-api/refresh`	operator+	Re-fetch one endpoint
PATCH	`/api/json-api/endpoint/poll`	integrator+	Enable/disable poll interval
DELETE	`/api/json-api/endpoint/{point_id}`	integrator+	Remove endpoint

Full route list: API routes.

50.7 Typical OT URLs

Device type	Example URL	Notes
Web weather		Env-based `appid`; 3

Device type	Example URL	Notes
	`https://api.openweathermap.org/data/2.5/weather?...`	historian columns
Gateway status	`http://192.168.1.50/api/status`	Often plain HTTP on LAN
HTTPS BMS API	`https://10.0.0.12/rest/points/zone1`	May need `verify_tls: false`
Local mock	`https://jsonplaceholder.typicode.com/todos/1`	Bench / CI only

50.8 Limitations

- Response body must be **JSON** (not XML or plain text).
- Numeric paths (e.g. `main.temp`) are stored as strings in poll CSV; FDD rules cast with `pc.cast(..., "float64")`.
- Outbound requests originate from the **bridge** container/host — ensure routing and firewall rules allow the edge box to reach the target (OT subnet or public internet for weather).

50.9 Disable polling (save API quota)

Stop scheduled calls without deleting endpoints: JSON API tree → select OpenWeather points → **Stop polling**, or integrator API:

```
curl -X PATCH http://127.0.0.1:8765/api/json-api/endpoint/poll \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"point_id":"ja-api-openweathermap-org-get-web-oat-t","enabled":false,"poll_interval_s":0}'
```

API keys live in `json_api.env.local` (gitignored) — never committed; audit logs do not record expanded URLs with `appid=`. See Logging and audit.

50.10 Smoke test

```
python3 scripts/smoke_multi_driver_stack.py
```

Validates JSON API ingest alongside BACnet and Modbus, feather isolation, BRICK scope, and FDD batch.

51 Algorithms

52 Algorithms (coming soon)

The **Algorithms** dashboard tab (`/algorithms`) is a placeholder for **supervisory Python sequences** — the mirror image of Rule Lab FDD:

	FDD (Rule Lab)	**Algorithms (planned)**
Purpose	Detect faults	Compute setpoints, request levels, plant enables
Entrypoint	`apply_faults_arrow(table, cfg, context)`	`apply_algorithm_arrow(table, cfg, context)` (planned)
Output	Boolean fault mask	Numeric outputs / structured dict
Historian	Same `feather_store` PyArrow tables	Same

Until the tab ships, draft patterns live in this doc and in GL36 algorithm stubs.

52.1 GL36 Trim & Respond reference

Open-FDD algorithms will follow the same ASHRAE Guideline 36 trim & respond methodology already implemented for Niagara in:

README_TRIM_RESPOND.md

That guide covers:

- VAV zone request generators (cooling + static pressure)
- AHU duct static pressure reset
- AHU supply air temperature reset
- Central plant chiller / boiler enable and HWST / CHWST trim & respond

Porting those blocks to PyArrow is in progress; the dashboard tab shows **COMING SOON** until upload, batch, and BACnet write paths are wired.

52.2 Planned workflow (same kit shape as Rule Lab)

1. Download **algorithm kit** zip from the Algorithms tab (future)

2. Edit **constants** at the top of `algorithm.py` (no `config.json`)
3. Run `python run_test.py` locally against `sample.feather`
4. Upload `algorithm.py` when satisfied

Kit files will match the Rule Lab zip layout (see Rule Lab — dev kit zip).

52.3 Lookback windows

Rules and algorithms that need multi-hour history should use the shared **lookback stats helper** documented in Lookback window helper. Pass `hours=1`, `3`, `6`, `12`, or `24` to print row count, timestamp start/stop, and span for console validation.

Scheduled batch defaults: **1 h** lookback on the FDD loop; **Update all records** in Rule Lab uses **24 h** with 6 h chunks.

52.4 AHU supervisory checks (doc-only stubs)

These are **not** uploaded rules yet — copy-paste references for future algorithm or FDD work.

52.4.1 Duct static pressure too high

Flags sustained high duct static (possible stuck setpoint, blocked filter, or trim/respond not trimming).

```
        """AHU duct static high — doc stub (Arrow constants)."""

        import pyarrow.compute as pc

        VALUE_COLUMN = "duct-static"
        HIGH_INWC = 1.20
        LOOKBACK_HOURS = 3
        MIN_SAMPLES_ABOVE = 6


        def _kit_lookback_stats(table, *, hours=None):
            h = hours if hours is not None else LOOKBACK_HOURS
            ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
            tmin = pc.min(ts).as_py()
            tmax = pc.max(ts).as_py()
            span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
            print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")
```

```
        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table)
            vals = pc.cast(table[VALUE_COLUMN], "float64")
            high = pc.greater(vals, HIGH_INWC)
            # Require several consecutive highs (~30 min at 5 min poll)
            from open_fdd.arrow_runtime.windows import
arrow_consecutive_true

            streak = arrow_consecutive_true(high, MIN_SAMPLES_ABOVE)
            return streak
```

52.4.2 Supply air temperature too cold (cooling coil over-delivering)

```
        """AHU SAT too cold vs setpoint — doc stub."""

        import pyarrow.compute as pc

        SAT_COLUMN = "sa-t"
        SP_COLUMN = "sa-t-sp"
        COLD_DELTA_F = 5.0
        LOOKBACK_HOURS = 1


        def _kit_lookback_stats(table, *, hours=None):
            h = hours if hours is not None else LOOKBACK_HOURS
            ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
            tmin = pc.min(ts).as_py()
            tmax = pc.max(ts).as_py()
            span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
            print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")


        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table)
            sat = pc.cast(table[SAT_COLUMN], "float64")
            sp = pc.cast(table[SP_COLUMN], "float64")
            return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)
```

52.5 Plant supervisory checks (doc-only)

See GL36 algorithm stubs for chiller plant enable, HWST trim & respond, and CHW DP/CHWST reset patterns aligned with the Niagara README.

52.6 Related

- Rule Lab — live FDD authoring
- Model workflow — point bindings
- Windowing & debugging — rolling windows

53 Model workflow

54 Model workflow

54.1 Brick model

- **Equipment tree** — SPARQL over synced TTL (`workspace/data/data_model.ttl`)
- **Point registry** — BACnet inventory → `series_id` for historian bindings
- **Health score** — orphan points, missing `fdd_input`, stale telemetry

54.2 Typical integrator flow

1. Import or sync BACnet point registry.
2. Map equipment classes (AHU, VAV, …) in the model UI.
3. Tag points with `fdd_input` roles (zone temp, SAT, damper cmd, …).
4. Bind Rule Lab rules to those inputs.
5. Run `POST /api/model/sync-ttl` after bulk imports.

Export: `GET /api/model/export` (TTL/JSON). API detail in Appendix.

55 Driver framework

56 Driver framework

Open-FDD edge commissioning uses a **shared driver pattern** for OT data sources. Each driver polls field devices on a schedule, stores long-format samples in workspace CSVs, and ingests into the **feather historian** under a distinct `source` tag for FDD and plots.

56.1 Drivers

Driver	Protocol	Feather `source`	Commissioning tab
BACnet	MS/TP / IP (: `47808`)	`bacnet`	BACnet
Modbus	Modbus TCP	`modbus`	Modbus
JSON API	HTTP/HTTPS REST (+ OpenWeather showcase)	`json_api`	JSON API

All three drivers share the same operator workflow:

1. **Discover or configure** — add devices/endpoints to the driver registry.
2. **Request once** — on-demand read to verify connectivity and JSON path / register map.
3. **Request & store** — append to poll CSV and write a feather shard.
4. **Enable polling** — 1 / 5 / 10 / 15 minute intervals via tree right-click or bulk toolbar.
5. **Poll all now** — force one worker cycle without waiting for the interval.

56.2 Data layout

```
workspace/
  bacnet/commissioning/   points.csv, discovered inventory
  bacnet/polls/           samples.csv
  modbus/commissioning/   registers.csv
  modbus/polls/           samples.csv
  json_api/commissioning/ endpoints.csv
  json_api/polls/         samples.csv
```

```
data/feather_store/
  bacnet/{site_id}/     shard-*.feather
  modbus/{site_id}/
  json_api/{site_id}/
```

56.3 Historian and FDD

- **Plot tab** — numeric columns from `bacnet` and `modbus` sources (`GET /api/timeseries/plot?source=…`).
- **String JSON fields** — stored in feather under `json_api`; use the JSON API tree or feather directly (plot is numeric-only).
- **FDD batch** — merges `bacnet` + `modbus` + `json_api` historian columns by nearest timestamp; rules use BRICK-bound `external_id` names (e.g. compare BACnet `oa-t` vs JSON API `web-oat-t` — see JSON API).
- **BRICK model** — bind points in **Model & assignments** commissioning JSON; sync TTL for SPARQL scope.

56.4 Rule Lab (Arrow upload/download)

1. **Download kit** — zip with `rule.py` (constants at top), `data.py`, `sample.feather` (~3h), `run_test.py`, `requirements.txt`.
2. Local test: `pip install -r requirements.txt` then edit constants in `rule.py` and `python run_test.py`.
3. **Upload rule.py** — Arrow-only; must define `apply_faults_arrow(table, cfg, context=None)`.
4. Browser shows **read-only** source; use Quick test + Update all records.

API: `GET /api/rules/export-kit`, `POST /api/rules/upload`.

56.5 Smoke validation

From repo root on a running stack (`http://127.0.0.1:8765`):

```
python3 scripts/smoke_multi_driver_stack.py
python3 scripts/validate_modbus_temp_e2e.py --no-server
```

56.6 Related docs

- Data flow — poll → CSV → feather → FDD
- API routes — REST reference
- BACnet polling — commission container loop

57 Operator Bridge

58 Operator Bridge

The Operator Bridge is the FastAPI service and React dashboard operators use daily.

Page	Topic
Dashboard overview	Main UI areas
Auth and roles	Login and permissions
Rule Lab	Python FDD rule authoring (PyArrow kit zip)
Algorithms	Supervisory GL36 sequences (coming soon)
Model workflow	Brick bindings and imports

REST details: Appendix — API routes.

59 Network setup

60 Network setup

60.1 Bind address

Commission and poll containers need the **OT interface** IP and UDP **47808**:

```
# workspace/bacnet/commissioning/commission.env
BACNET_BIND=192.168.1.50/24:47808
BACNET_INSTANCE=599999
```

- Use the NIC that can reach controllers (not only the management VLAN unless routed).
- Poll driver uses `network_mode: host` on Linux edges.

60.2 Discovery range

```
DISCOVER_LOW=1
DISCOVER_HIGH=4194302
DISCOVER_TIMEOUT=30
```

Narrow the range in large sites to avoid broadcast storms.

60.3 Verify

From commission container logs or API: Who-Is → I-Am responses. If zero devices, check firewall, bind IP, and VLAN routing before tuning rules.

61 Discover and read

62 Discover and read

The **commission** container exposes HTTP APIs for discovery and read-property workflows used by the dashboard BACnet tools.

Capability	Notes
Who-Is / I-Am	Device discovery on OT subnet
Read property	Present value, object name, units
RPM	Bulk reads for inventory export
Write property	**Gated** — see Write safety

Exported inventory feeds `points_discovered.csv` and the Brick point registry.

62.1 Operator workflow

1. Confirm network setup.
2. Run discover from dashboard or API.
3. Review discovered objects; enable points for poll profiles.
4. Sync model bindings for FDD inputs.

63 Polling

64 Polling

BACnet polling has two stages:

1. **Field reads** — commission poll loop → `workspace/bacnet/polls/samples.csv`
2. **Historian ingest** — bridge background worker → `workspace/data/feather_store/`

64.1 Commission poll loop (default)

The **commission** container runs the poll loop at startup. It reads enabled rows from `points.csv` and appends BACnet RPM results to `samples.csv`.

Setting	Location
BACnet bind	`workspace/bacnet/commissioning/commission.env`
Enabled points	`workspace/bacnet/commissioning/points.csv`
Poll output	`workspace/bacnet/polls/samples.csv`
Historian	`workspace/data/feather_store/`

Check activity:

```
tail -1 workspace/bacnet/polls/samples.csv
docker compose logs --since 5m commission | grep -i poll
```

64.2 Bridge ingest worker

Inside **bridge**, a background thread watches `samples.csv` and ingests new rows into feather. Disable only for debugging: `OFDD_DISABLE_POLL_WORKER=1`.

64.3 Health

Dashboard stack health (`GET /health/stack`) reports `bacnet_poll` status from the commission agent — not a separate container.

Stale points trigger data-quality fault patterns — see Sensor quality faults.

65 Write safety

66 Write safety

> **BACnet writes can change live plant behavior.** OpenFDD disables writes by default. Enable only with operator sign-off, a tested allowlist, and audit logging.

66.1 Defaults

Control	Default
Supervisory writes	**Off**
Write allowlist	Empty — must enumerate device/object
Audit	Commands logged when writes enabled

66.2 Enable writes (commission role)

1. Set env flag documented in Security (`OFDD_BACNET_WRITE_ENABLED` or site equivalent).
2. Populate write allowlist CSV / config with explicit object IDs.
3. Use **commission** role JWT; every write should include reason/note where API supports it.
4. Test on bench hardware before production OT.

66.3 Operator rules

- Never write setpoints or overrides on life-safety or fire/smoke interlocks through experimental rules.
- Prefer read-only polling for FDD until rules are validated in Rule Lab.
- Revert overrides through BACnet priority relinquish where supported.

67 BACnet

68 BACnet

Open-FDD integrates with field controllers via BACnet/IP (and bench MSTP overlays in dev compose).

Page	Topic
Network setup	Bind address and NIC
Discover and read	Commission agent
Polling	Historian driver
Write safety	Defaults and allowlists

69 Convention

70 Fault code convention

70.1 Format

`FAMILY-SUFFIX` — suffix is **1–3 letters** (e.g. `VAV-C`, `AHU-B`). Numeric suffixes like `VAV-03` are avoided (collision with physical equipment names).

70.2 Categories

Category	Meaning
`performance_degradation`	Efficiency or capacity drift
`simultaneous_heat_cool`	Heating and cooling fighting
`sensor_fault`	Flatline, OOB, inconsistent readings
`io_fault`	Command vs feedback mismatch

70.3 Severity

Level	Operator response
`info`	Log; trend for maintenance window

Level	Operator response
`warning`	Investigate within days
`critical`	Prompt attention; may affect comfort/energy significantly

70.4 Linking rules

In Rule Lab, set `fault_code` on the rule metadata. Batch FDD aggregates hits into `GET /api/faults/status` by family.

70.5 Cookbook mapping

Each catalog entry lists `cookbook_patterns` (e.g. `flatline_1h`) — see Rule Cookbook.

71 AHU faults

72 AHU faults

Air handling unit codes (sample from catalog).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-A	Supply fan performance degradation	warning	Spread/ runtime vs baseline	Belt slip, dirty filters, VFD limit	Trend speed vs airflow; filter dP
AHU-B	Simultaneous heating and cooling	critical	Heat + cool outputs active together	Sequencing bug, leaking valve	Trend coil commands; valve feedback
AHU-C	SAT sensor fault	warning	Flatline / OOB SAT	Failed sensor, bad calibration	SAT trend; compare to coil discharge
AHU-D	MAT inconsistent with OAT/ RAT	warning	MAT outside OAT–RAT envelope	MAT/OAT/ RAT sensor errors	Verify damper positions; cross-check OAT

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-E	Economizer not economizing	warning	Free cool available, OA minimum	Stuck damper, lockout	OA damper vs OAT trend
AHU-F	Damper command vs feedback mismatch	warning	Cmd ≠ feedback	Stuck actuator, linkage	Compare cmd/ feedback trends

Example rule: `flatline_1h` on SAT → tag **AHU-C**. Recipe: Python recipes.

73 VAV faults

74 VAV faults

Variable air volume terminal codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
VAV-A	Zone heating/ cooling overlap	warning	Heat and cool at terminal	Reheat valve stuck, SAT too low	Zone temp vs SAT; valve cmd
VAV-B	Poor zone temperature control	warning	Persistent deviation from setpoint	Oversized box, bad tuning	PI loop trends; occupancy
VAV-C	Zone temperature sensor fault	warning	Flatline / OOB zone temp	Failed zone sensor	Flatline test; compare to neighbor zones
VAV-D	Minimum airflow not met	warning	Flow below minimum cmd	Damper stuck, pressure issue	Airflow vs damper cmd
VAV-E	Discharge temp	warning	DAT far from SAT setpoint	Coil issue,	DAT vs SAT SP

Code	Title	Severity	Detection summary	Likely causes	Operator checks
	deviation from SAT			sensor drift	

Example rule: `oob_rolling` on zone temp → **VAV-C** or comfort bounds per site policy.

75 RTU faults

76 RTU faults

Packaged rooftop unit codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
RTU-A	Compressor short cycling	warning	Rapid on/off cycles	Low charge, oversized unit	Run-time histogram
RTU-B	Economizer / OA damper fault	warning	OA behavior vs conditions	Actuator, sensor	OA damper vs enthalpy/ OAT
RTU-C	Supply air temperature fault	warning	SAT flatline or OOB	Sensor failure	SAT trend vs outdoor conditions
RTU-D	Simultaneous heat and cool	critical	Heat + cool stages together	Control board, relay stuck	Stage status trends

Bind rules to packaged unit `fdd_input` points (SAT, OAT, compressor cmd, …).

77 Sensor and data quality

78 Sensor and data quality

Code	Title	Severity	Detection summary	Likely causes	Operator checks
BLD-B	Outdoor air sensor fault	warning	OAT flatline / unrealistic	Sensor exposure, failed probe	Compare to weather service
BLD-D	Stale telemetry	warning	No new samples within threshold	Poll driver down, device offline	Poll health; BACnet comms
DC-C	Datacom sensor anomaly	warning	CRAH/ room sensor flatline	Failed sensor	Row-level flatline recipe

78.1 Communication quality

- Monitor poll cycle timestamps in dashboard.
- Use `stale_points` cookbook pattern for dropout detection.
- Do not confuse **comm loss** with **equipment fault** — verify BACnet device online before dispatching mechanical.

78.2 Alarm console quality

When integrating with BMS alarms (future/external), map Open-FDD codes to operator work orders — Open-FDD itself is **analytics-first**, not a dial-out alarm server.

79 Fault Codes

80 Fault Codes

Open-FDD uses a **check-engine** model: one building-level status (green / yellow / red) with a catalog of fixed fault codes per equipment family.

Page	Content
Convention	Naming, severity, categories
AHU faults	Air handling units
VAV faults	Terminal units
RTU faults	Packaged rooftops
Sensor & data quality	Stale, flatline, comms

Live catalog API: `GET /api/faults/catalog` on the Operator Bridge.

81 Live site update (SSH)

82 Live site update (SSH)

> **IT operators:** prefer Quick Start — Updating the stack and the helper scripts `openfdd_site_backup.sh` / `openfdd_site_update.sh`. This page is the detailed SSH runbook with manual commands.

Use this runbook when a **live edge VM** already has a minimal deploy folder and you are **not** doing `git pull` on the host.

Typical layout (no full git checkout):

```
/home/bbartling/open-fdd/
  docker/
  docker-compose.yml
  workspace/
```

`workspace/` is the **live site state** — BACnet commissioning files, poll CSV, feather historian, BRICK model, FDD rules, MCP RAG index, and env files. Image upgrades must **preserve** it.

82.1 Concepts (plain language)

Term	Meaning
Container	A running app instance (bridge, commission, mcp-rag). Recreated on upgrade.
Image	Downloaded app package from GHCR (e.g. `openfdd-bridge:2026.06.07-edge`).
Docker volume	Persistent data Docker manages separately. This minimal layout uses **bind mounts** into `workspace/` instead.
`workspace/`	Open-FDD site state on disk. **This is what you must not lose.**

> **Never on live sites:** `docker compose down -v`, `docker volume prune`, `docker system prune -a --volumes`, or `docker rm -f $(docker ps -aq)`. Those can destroy data or leave the site down with no rollback path.

82.2 Before you start

1. Maintenance window or low-traffic period (containers restart briefly).
2. SSH access to the edge host.
3. New GHCR tag published (check GitHub Packages).
4. Optional: control machine with the full `open-fdd` repo for post-deploy check.

82.3 1. Backup `workspace/`

Container processes may write files as **`root:root`** with mode **`0600`**. A normal user `tar` can fail on those paths — use `sudo`:

```
cd /home/bbartling/open-fdd

export BACKUP_ROOT="$HOME/openfdd-backups/$(date +%Y%m%d-%H%M%S)"
mkdir -p "$BACKUP_ROOT"

cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before"

docker compose ps > "$BACKUP_ROOT/docker-compose-ps-before.txt"
docker compose config --images > "$BACKUP_ROOT/docker-images-before.txt"

sudo tar --xattrs --acls -czf "$BACKUP_ROOT/workspace-full.tgz" workspace
sudo chown "$USER:$USER" "$BACKUP_ROOT/workspace-full.tgz"
```

```
        echo "Backup saved to: $BACKUP_ROOT"
        du -h "$BACKUP_ROOT/workspace-full.tgz"
```

82.4 2. Verify new images exist on GHCR

```
        export NEW_TAG="2026.06.07-edge"

        docker manifest inspect ghcr.io/bbartling/openfdd-bridge:$
{NEW_TAG} >/dev/null &&
        docker manifest inspect ghcr.io/bbartling/openfdd-commission:$
{NEW_TAG} >/dev/null &&
        docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:$
{NEW_TAG} >/dev/null &&
        echo "All images exist for ${NEW_TAG}"
```

Replace `NEW_TAG` with the tag you intend to deploy.

82.5 3. Update the React UI bundle (required for dashboard changes)

> **Image pull alone does not update the browser UI.** The bridge serves `workspace/api/static/app/` from the **bind mount first**, before anything baked into the GHCR image (`static_dashboard_dir()` in the bridge). If that folder still has an old `index-*.js` hash (e.g. `index-TRH4YIfA.js`), you will keep seeing the old BACnet tree, Host Stats revisions panel, Model & assignments tab, etc.

From bensserver (control machine, full repo):

```
        cd /path/to/open-fdd
        ./scripts/build_operator_dashboard.sh prod
        cd infra/ansible
        ./deploy.sh ui --limit acme_vm_bbartling
```

Confirm the edge picked up the new hash:

```
        curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.js' | head -1
        # Should match bensserver: grep -o 'index-[^"]*\.js' workspace/
api/static/app/index.html
```

One command (recommended after CI publishes a new tag):

```
        OPENFDD_IMAGE_TAG=2026.06.08-edge ./scripts/upgrade_edge_full.sh
--limit acme_vm_bbartling
```

That runs: dashboard build → `deploy.sh ui` → GHCR pull + `docker compose up -d --force-recreate` → `post_deploy_check.sh --full`.

82.6 4. Update `docker-compose.yml` and recreate containers

Edit image tags in `docker-compose.yml` (example: `2026.06.04-edge` → `2026.06.07-edge`):

```
        cd /home/bbartling/open-fdd

        cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before-tag-change"
        sed -i "s/2026\.06\.04-edge/${NEW_TAG}/g" docker-compose.yml

        docker compose config --images
        docker compose pull
        docker compose up -d --force-recreate

        docker compose ps
```

Bind-mounted `workspace/` is unchanged; only container images restart.

82.7 5. On-VM smoke checks

```
        cd ~/open-fdd

        echo "=== Stack ==="
        docker compose ps
        docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"

        echo "=== Bridge health ==="
        curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://127.0.0.1:8765/health

        echo "=== Stack health (if available — may require auth) ==="
        curl -i http://127.0.0.1:8765/health/stack

        echo "=== MCP health from inside mcp-rag container ==="
        docker compose exec mcp-rag sh -lc 'curl -sf http://127.0.0.1:8090/health || wget -qO- http://127.0.0.1:8090/health || python - <<PY
        import urllib.request
        print(urllib.request.urlopen("http://127.0.0.1:8090/health", timeout=5).read().decode())
        PY'

        echo "=== BACnet samples still moving ==="
```

```
        ls -lah workspace/bacnet/polls/samples.csv
        tail -n 1 workspace/bacnet/polls/samples.csv

        echo "=== BACnet ingest state ==="
        sudo cat workspace/data/bacnet_ingest_state.json | jq . || sudo
cat workspace/data/bacnet_ingest_state.json

        echo "=== Feather store retained and writing ==="
        du -sh workspace/data/feather_store
        find workspace/data/feather_store -type f -printf "%TY-%Tm-%Td
%TH:%TM %p\n" | sort | tail -n 10

        echo "=== Recent log errors ==="
        docker compose logs --since 20m | grep -Ei "error|exception|
traceback|critical|failed|permission|denied" || true
```

`/health/stack` nuance: `/health` should return `200` without auth. `/health/stack` may return `401`, `403`, `404`, or an empty body depending on auth settings — use `curl -i` (not `curl -sf`) so you see the status code. A failed silent `curl -sf` does not mean the stack is broken.

82.7.1 Arrow / FDD rules (Open-FDD 3.0+)

All saved rules use `apply_faults_arrow(table, cfg, context)` on PyArrow tables.

From bensserver:

```
        ./scripts/validate_fdd_backends.sh --docker   # must exit 0
```

On the edge VM after upgrade:

```
        grep -R "apply_faults_arrow" -n workspace/data/rules_py 2>/dev/
null | wc -l
        docker compose exec bridge python3 -c "
        from openfdd_bridge.rule_store import RuleStore
        from open_fdd.arrow_runtime.rules import detect_rule_backend
        from openfdd_bridge.rule_source import read_source
        for r in RuleStore().list_rules():
            if not r.get('enabled', True): continue
            code = read_source(r.get('source_path','')) or
r.get('code','')
            print(r.get('name'), detect_rule_backend(code, r))
        "
```

Spot-check **Rule Lab** quick-test shows `backend: arrow`.

82.8 6. Control-machine insurance check

From your laptop or bensserver (full repo clone, not on the edge VM):

```
cd /path/to/open-fdd/infra/ansible

./scripts/post_deploy_check.sh --limit acme_vm_bbartling

# Or explicitly:
./scripts/post_deploy_check.sh --host 100.122.106.124 --ssh-user
bbartling
```

This is the recommended **non-destructive** check: Caddy → React dashboard, `/health/stack`, MCP, BACnet tree, BRICK/SPARQL, Docker log scan.

> **Do not run on live commissioned sites:**
> `openfdd_edge_validate.sh --full` or `acme_operational_verify.sh` — those rediscover BACnet and reset bench model/rules.

82.9 7. Browser validation

Open the site via **Caddy on port 80** (e.g. `http://<tailscale-or-lan-ip>/`), not only `127.0.0.1:8765`:

1. View page source / Network — confirm **new** `index-*.js` hash (not the pre-upgrade bundle).
2. Home / faults dashboard loads.
3. Integrator login (`workspace/auth.env.local`).
4. **Host stats** — container image tag / git sha (3.0+ bridge).
5. **BACnet** — right-click point → **Refresh PV**, **Read priority array**.
6. **Model & assignments** (`/model`) — commissioning JSON export (404 = bridge image + UI both stale).
7. Rule Lab — quick-test a saved rule (`backend: arrow`).

82.10 Rollback

If the new tag misbehaves, restore the previous tag in compose and recreate:

```
cd /home/bbartling/open-fdd

# Example: roll back from 2026.06.07-edge to 2026.06.04-edge
export NEW_TAG="2026.06.07-edge"
export OLD_TAG="2026.06.04-edge"
```

```
        sed -i "s/${NEW_TAG}/${OLD_TAG}/g" docker-compose.yml

        docker compose pull
        docker compose up -d --force-recreate

        docker compose ps
        curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health
```

`workspace/` is unchanged by rollback. For data corruption, extract files from `$BACKUP_ROOT/workspace-full.tgz` selectively (model, rules, feather shards) — do not blindly overwrite a running site without stopping containers first.

82.11 Alternative: Ansible image-only upgrade

If you have inventory and SSH from a control machine (no manual `sed` on the VM):

```
        export OPENFDD_IMAGE_TAG=2026.06.07-edge
        RUN_POST_CHECK=1 ./scripts/upgrade_edge_ghcr.sh --limit
acme_vm_bbartling
```

See also Updating the stack.

83 LAN hardening

84 LAN hardening

84.1 Deployment modes

Mode	Bind	Auth
Local dev	`127.0.0.1`	Optional `OFDD_AUTH_DISABLED=1`
Lab LAN (insecure)	`0.0.0.0`	Requires explicit insecure dev flags — **not production**
Edge production	`0.0.0.0` behind Caddy	**Required** credentials + `OFDD_AUTH_SECRET`

Startup **fails** on non-loopback bind without credentials (unless insecure lab flags are explicitly set).

84.2 Network exposure

- Do not port-forward the bridge to the public internet without TLS and strong auth.
- Keep BACnet on OT VLANs; bridge management on IT VLAN with firewall rules.
- Prefer Caddy on `:80/443` rather than exposing `:8765` directly.

84.3 Rule Lab caution

Rule Lab executes **operator-authored Python** in a sandbox. Restrict integrator accounts; review rules before enable on production sites.

85 Secrets and auth

86 Secrets and auth

86.1 Files (gitignored)

File	Contents
`workspace/auth.env.local`	JWT secret, role passwords
`workspace/json_api.env.local`	REST API keys for JSON API `${ENV:VAR}` URLs (e.g. OpenWeather `OPENWEATHER_API_KEY`)
`workspace/data.env.local`	Historian + **audit log retention** (`OFDD_LOG_*`, `OFDD_FEATHER_*`)
`infra/ansible/secrets/<host>.env.local`	SSH deploy secrets (maintainers)

Copy from `*.example` templates only.

86.2 Required variables (production)

Variable	Purpose
`OFDD_AUTH_SECRET`	HMAC signing (32+ chars)
`OFDD_INTEGRATOR_USER` / `PASSWORD`	Rule Lab, bindings
`OFDD_OPERATOR_USER` / `PASSWORD`	Operations

86.3 GHCR pulls

Use read-only registry credentials on edge if images are private; rotate tokens periodically.

86.4 Backup

Back up `workspace/data/` and auth env in secure operator vault — not in git.

Audit JSONL under `workspace/logs/` may contain client IPs and usernames — treat like security logs. See Logging and audit.

87 BACnet write allowlists

88 BACnet write allowlists

Supervisory writes require:

1. **Feature flag** enabled in environment (off by default).
2. **Allowlist** file at `workspace/bacnet/write_allowlist.json` (copy from `write_allowlist.example.json`). If the flag is on but the allowlist is missing, writes are **denied**.
3. **Integrator** role on API calls.
4. **Audit** log review after commissioning sessions.

Allowlist entries support `device_instance`, `object_identifier`, `property_identifier`, optional `priority_min`/`priority_max`, `value_min`/`value_max`, and `allowed_values` for binary/multistate targets.

Lab-only override: `OFDD_BACNET_WRITE_ALLOW_ANY=1` (audited; not for production edges).

See operator guide: BACnet write safety.

Never enable blanket write access for agent or integrator automation without human approval per site.

89 Logging and audit

90 Logging and audit

Open-FDD uses a **two-tier logging model** familiar to IT and security teams: structured **audit/error JSONL** for forensics and compliance, plus **service stdout** for engineering troubleshooting. Defaults include **age pruning**, **size rotation in MB**, and **Docker log caps** — no operator cron required.

90.1 Log types

Tier	Location	Format	Who reads it
Audit	`workspace/logs/audit.jsonl`	JSON Lines	Security, MSI, integrator API
Errors	`workspace/logs/error.jsonl`	JSON Lines	Support, integrator API
Service stdout	Docker: `docker compose logs` · Dev: `workspace/.local-run/*.log`	Plain text (uvicorn, workers)	Engineering
Historian	`workspace/data/feather_store/`	Feather	FDD, plots — separate retention (`data.env.local`)

Structured audit logs are **not** mixed into uvicorn access logs. That separation matches common practice: SIEM-friendly JSONL for auth and OT commands; unstructured stdout for stack traces and poll worker noise.

90.2 Audit record schema

Each line in `audit.jsonl` / `error.jsonl` is one JSON object:

Field	Example	Notes
`@timestamp`	`2026-06-07T19:57:21+00:00`	UTC ISO-8601
`event_id`	UUID	Unique per event

Field	Example	Notes
`event_type`	`auth.login.failure`	Stable taxonomy (see below)
`severity`	`warning`	`debug` … `critical`
`outcome`	`failure`	`success`, `failure`, `denied`, …
`service`	`openfdd-bridge`	
`host`	hostname	
`action`	`login`	Human-readable verb
`actor`	`{username, role}`	After successful auth
`client`	`{ip, user_agent}`	Respects `OFDD_TRUST_X_FORWARDED_FOR` behind Caddy
`http`	method, path, status	When HTTP-triggered
`resource`	BACnet object, model id, …	When applicable
`detail`	Sanitized map	**Passwords, tokens, secrets stripped by key name**
`request_id`	UUID	On audited HTTP paths

90.2.1 Auth events (industry-standard login trail)

`event_type`	When
`auth.login.success`	Valid credentials
`auth.login.failure`	Bad password (generic client message)
`auth.login.lockout`	Rate limit exceeded (default 5 failures / 5 min)

Login rate limits and lockouts are **audited**; responses never reveal whether the username exists.

90.2.2 OT / commissioning events

`event_type`	When
`bacnet.command`	Supervisory write (allowed, denied, dry-run)
`bacnet.discover`	Who-Is / point discovery
`model.write`	BRICK model mutations
`agent.tool`	Agent tool calls (prompts hashed/truncated)
`api.access`	

event_type	When
	Sensitive POST/PUT/PATCH/DELETE (BACnet, ingest)
`error.unhandled`	Unhandled exception (also in `error.jsonl`)

Not audited: routine GETs (dashboard reads, plot data, health). This reduces noise while keeping **auth and field commands** on the trail.

90.2.3 Secret redaction

Never logged: passwords, bearer tokens, `OFDD_AUTH_SECRET`, private keys. Detail objects drop keys containing `password`, `token`, `secret`, `authorization`. Agent tool arguments use additional hashing for prompts and rule source.

Opt-in full prompt logging (lab only): `OFDD_AUDIT_LOG_PROMPTS=1`.

90.3 Integrator API (read logs in the UI stack)

Method	Path	Role
GET	`/api/audit/events?limit=100`	integrator
GET	`/api/audit/errors?limit=100`	integrator
GET	`/api/audit/summary`	integrator

Use these for post-incident review without shell access. Forward `workspace/logs/*.jsonl` to SIEM (rsyslog, Filebeat, Wazuh) on production edges.

90.4 Retention and rotation (defaults)

Configured in `workspace/data.env.local` (copy from `data.env.example`):

Variable	Default	Behavior
`OFDD_LOG_RETENTION_DAYS`	**90**	Drop audit/error JSONL lines older than N days; delete aged `.local-run` and archived `*.log` / `audit.*.jsonl`
`OFDD_LOG_MAX_MB`	**50**	When `audit.jsonl` or `error.jsonl`

Variable	Default	Behavior
		exceeds N MB, archive to `audit.{UTC}.jsonl` and start fresh
`OFDD_LOCAL_RUN_LOG_MAX_MB`	**25**	Rotate `workspace/.local-run/*.log` (bridge, Caddy, FDD loop, …)
`OFDD_LOG_ROTATE_INTERVAL_HOURS`	**6**	Background retention while bridge is running (not only at restart)

Rotation runs at **bridge startup** and on the **scheduled interval**. No logrotate.d entry is required for audit JSONL.

Optional path overrides:

```
OFDD_AUDIT_LOG_PATH=/var/log/openfdd/audit.jsonl
OFDD_ERROR_LOG_PATH=/var/log/openfdd/error.jsonl
```

90.5 Docker edge: json-file caps (not journald for app audit)

Production stacks use **Docker Compose** (`docker/compose.edge.yml`). Container stdout uses the **`json-file` driver with size limits**:

```
logging:
  driver: json-file
  options:
    max-size: "25m"
    max-file: "5"
```

That caps each service at roughly **125 MB** of rotated container logs on disk — the same pattern many teams use instead of unbounded `docker logs`.

Why audit JSONL stays on disk (not journald)?

- Append-only files under `workspace/logs/` survive container recreate and bind-mount to the host.
- JSONL lines import cleanly into SIEM without journal field parsing.

- OT incident response often needs **months** of auth/BACnet write history on the edge box.

When journald still applies: native systemd units (Caddy, Ollama on bare metal) — use `journalctl -u <unit>` per your Linux runbook. Bridge audit remains in `workspace/logs/` even when uvicorn stdout goes to Docker or `.local-run/`.

90.6 Local dev (`run_local.sh`)

File	Service
`workspace/.local-run/bridge.log`	Bridge / uvicorn
`workspace/.local-run/commission.log`	BACnet commission agent
`workspace/.local-run/caddy.log`	Caddy
`workspace/.local-run/fdd_loop.log`	Scheduled FDD
`workspace/.local-run/mcp_rag.log`	MCP RAG
`workspace/.local-run/ollama.log`	Ollama

These are **engineering logs**. Structured audit still lands in `workspace/logs/audit.jsonl` when auth is enabled.

90.7 Quick checks

```
# Last auth failures
grep 'auth.login' workspace/logs/audit.jsonl | tail -5

# Integrator API (after login)
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8765/api/audit/summary | jq .

# Docker edge
docker compose logs bridge --tail 50
```

90.8 Comparison to typical web apps

Practice	Open-FDD
Structured auth audit	Yes — JSONL with actor, IP, outcome
Login rate limit + lockout audit	Yes
Secret redaction in logs	Yes — key-based + agent sanitization
Request correlation ID	

Practice	Open-FDD
	Yes — on audited mutation paths
Centralized stdout JSON	No — stdout is plain text; audit is JSONL
Full HTTP access log	No — mutations and auth only (OT noise reduction)
Log rotation / size caps	Yes — defaults above + Docker `max-size`
SIEM-ready export	Yes — file tail / Filebeat on `workspace/logs/`

For BACnet write safety and credential generation, see SECURITY.md and Secrets and auth.

90.9 Related

- JSON API driver — OpenWeatherMap showcase, env-based API keys (`json_api.env.local`)
- Live site update — `docker compose logs` after upgrades
- Data flow — historian retention (`OFDD_FEATHER_*`) is separate from audit logs

91 Acme GL36 FDD (vm-bbartling)

92 Acme GL36 FDD (vm-bbartling)

Doc-only reference for **ASHRAE Guideline 36** supervisory monitoring on the Acme edge (`acme` / `vm-bbartling`). Rule code lives in `workspace/data/rules_py/`; commissioning uses **Arrow constants** (no `config.json`).

Trim & respond Niagara reference: README_TRIM_RESPOND.

92.1 BACnet poll scope

Device filter (`infra/ansible/scripts/acme_trim_devices.sh`):

Range	Equipment
1–100	JCI VMA/VAV
1100	RTU-01 (AHU)
1002	Hot water plant
11000–13000	Trane UC210 VAV

Poll interval: **60 s**. Commission: `POLL_INTERVAL=60 ./infra/ansible/scripts/acme_commission_gl36.sh` → `edge_backup/local/acme/vm-bbartling/points.csv`.

92.2 Example FDD rules (Arrow constants)

92.2.1 Duct static pressure high (AHU)

```
        """Acme AHU duct static high — GL36 trim advisory."""

        import pyarrow.compute as pc

        VALUE_COLUMN = "duct-static"
        HIGH_INWC = 1.20
        LOOKBACK_HOURS = 3


        def _kit_lookback_stats(table, *, hours=None):
            h = hours if hours is not None else LOOKBACK_HOURS
            ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
            tmin, tmax = pc.min(ts).as_py(), pc.max(ts).as_py()
            span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
            print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")


        def apply_faults_arrow(table, cfg, context=None):
            _kit_lookback_stats(table)
            return pc.greater(pc.cast(table[VALUE_COLUMN], "float64"),
HIGH_INWC)
```

92.2.2 SAT too cold vs setpoint

```
        """Acme AHU SAT too cold vs setpoint."""

        import pyarrow.compute as pc

        SAT_COLUMN = "sa-t"
        SP_COLUMN = "sa-t-sp"
```

```
COLD_DELTA_F = 5.0


def apply_faults_arrow(table, cfg, context=None):
    sat = pc.cast(table[SAT_COLUMN], "float64")
    sp = pc.cast(table[SP_COLUMN], "float64")
    return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)
```

92.2.3 Zone temp flatline (existing site rule)

See `workspace/data/rules_py/bench_stat_zn-t_flatline_1h.py` — bind `ZN-T` per VAV in Model & assignments.

92.2.4 Duct static flatline (site rule stub)

See `workspace/data/rules_py/acme_duct_static_flatline_1h_gl36_duct_t_r_input.py` — tune `VALUE_COLUMN` to historian column for RTU duct static.

92.2.5 Local OA-T vs web weather (JSON API)

Cross-check a drifting outdoor-air sensor against OpenWeatherMap via the generic JSON API driver:

1. Control machine: `workspace/json_api.env.local` with `OPENWEATHER_API_KEY` — synced on deploy (`deploy_sync_json_api_env`).
2. JSON API tab → **Register OpenWeather bundle** (poll **20 min** — `web-oat-t`, `web-rh`, `web-weather-desc`).
3. Bind BACnet **local OAT** in the model: **Tracer SC** device `10000` → `Facility Outdoor Air Temperature` (`fdd_input` **OAT**, historian `oa-t`). Hot-water plant `1002` has no outdoor-air point.
4. Enable rule `workspace/data/rules_py/oat_vs_web_spread_1h.py` in Rule Lab — flags when `|oa-t − web-oat-t| > 8 °F`.

Scheduled FDD merges BACnet and `json_api` historian columns by nearest timestamp (30 min tolerance).

92.3 Deploy / upgrade (GHCR only)

```
cd infra/ansible
POLL_INTERVAL=60 ./scripts/acme_commission_gl36.sh
OPENFDD_IMAGE_TAG=latest ./deploy.sh docker --limit
acme_vm_bbartling -e enable_bacnet_poll_driver=true
```

Do **not** rsync dev `auth.env.local` to the edge (`deploy_sync_auth: false` in `host_vars/acme_vm_bbartling.yml`). Regenerate on-host if the bridge rejects example credentials.

92.4 AI-assisted data modeling

1. BACnet poll running (340+ GL36 points @ 60 s).
2. **Model & assignments** → copy prompt + commissioning JSON for LLM.
3. Import returned JSON via commissioning import.
4. Pin `fdd_rule_ids` on `ZN-T`, `DA-T`, `duct-static`, `OAT`, etc.

Model seed: `workspace/data/acme_gl36_model.json`.

93 Operations

94 Operations

Runbooks for live edge hosts after the initial deploy.

Page	When to use
Logging and audit	Audit JSONL, auth trail, rotation defaults, Docker log caps, SIEM export
Live site update (SSH)	Minimal `~/open-fdd/` folder on a VM — pull new GHCR tags, preserve `workspace/`
Acme GL36 FDD	vm-bbartling poll scope, example Arrow rules, AI modeling workflow

For first-time deploy from a control machine, see Quick Start — Docker.

95 Security

96 Security

Concise deployment security for the Operator Bridge and BACnet tools.

Page	Topic
LAN hardening	Auth and bind modes
Secrets and auth	Env files
BACnet write allowlists	Supervisory write gates

Page	Topic
Logging and audit	Auth audit trail, rotation, SIEM

97 API routes

98 API routes

REST API served by **`openfdd-bridge`** (default `http://127.0.0.1:8765`). Production: Caddy on `:80` proxies to the bridge.

OpenAPI: `GET /docs` and `GET /redoc` when the bridge runs.

Auth: JWT via `POST /api/auth/login`. Most routes require `Authorization: Bearer <token>`. Roles: `viewer`, `operator`, `integrator`, `commission`, `agent`, `admin`. BACnet writes require commission role + write safety. Dev: `OFDD_AUTH_DISABLED` on loopback only.

WebSocket: `WS /ws/dashboard` — `POST /api/auth/ws-ticket`, then pass the ticket via `Sec-WebSocket-Protocol: ofdd.ws, <ticket>`. Query `?ticket=` is dev-only when `OFDD_WS_ALLOW_QUERY_TICKET=1`. Tickets are short-lived and not interchangeable with the Bearer token.

98.1 Health & audit

Method	Path	Role	Description
GET	`/health`	public	Minimal liveness (`ok`, `service`, `version`, `auth_required`)
GET	`/health/stack`	auth	Stack traffic-light (verbose URLs/bind with `OFDD_DEBUG_DIAGNOSTICS`)
POST	`/api/auth/ws-ticket`	auth	Short-lived WebSocket ticket
GET	`/api/audit/summary`	auth	Audit counters
GET	`/api/audit/events`	auth	Audit log (query params)

Method	Path	Role	Description
GET	`/api/audit/errors`	auth	Recent API errors

*Exact public set depends on `OFDD_AUTH_DISABLED` and middleware.

98.2 Auth

Method	Path	Description
POST	`/api/auth/login`	Username/password → JWT
GET	`/api/auth/me`	Current user
GET	`/api/auth/status`	Auth mode / lockout hints

98.3 Rules & FDD

Method	Path	Role	Description
GET	`/api/rules/saved`	user	Rule index
POST	`/api/rules/save`	operator+	Create/update rule metadata
GET	`/api/rules/saved/{id}/source`	user	Python source
PUT	`/api/rules/saved/{id}/source`	operator+	Write `.py` file
GET	`/api/rules/assignments`	user	Point bindings
POST	`/api/rules/bind`	integrator+	Bind rule to point
POST	`/api/rules/bindings`	integrator+	Bulk bindings
POST	`/api/rules/batch`	operator+	Run all enabled rules → `fdd_results.json`
GET/POST	`/api/rules/drafts`	user	Draft rules

98.4 Rule Lab playground

Method	Path	Description
POST	`/api/playground/lint`	AST lint Python rule
POST	`/api/playground/test-rule`	Run `apply_faults_arrow()` on feather PyArrow window
POST	`/api/playground/run-script`	Execute script mode
GET	`/api/playground/sample-frame`	Sample DataFrame for editor

98.5 BRICK / data model

Method	Path	Description
GET	`/api/model/sites`	Site list
GET	`/api/model/tree`	Equipment tree (SPARQL)
GET	`/api/model/graph`	RDF graph fragment
GET	`/api/model/scope`	Scoped query
GET	`/api/model/export`	Export TTL/JSON
GET	`/api/model/health`	Model point/equipment counts
GET	`/api/model/ttl`	TTL metadata
POST	`/api/model/sync-ttl`	Regenerate TTL from JSON
POST	`/api/model/import`	Import model payload
POST	`/api/model/sites`	Create site
GET/POST	`/api/model/bacnet-sync`	BACnet ↔ model sync state

98.6 BACnet

Method	Path	Role	Description
GET	`/config/bacnet`	read	Commission URL / config
GET	`/api/bacnet/`	read	

Method	Path	Role	Description
	`commission/status`		Commission agent health
GET	`/api/bacnet/server/points`	read	Server point table
GET	`/api/bacnet/inventory`	read	Discovered inventory
POST	`/api/bacnet/whois`	commission	Who-Is
POST	`/api/bacnet/discover`	commission	Discover devices
POST	`/api/bacnet/read`	commission	Read property
POST	`/api/bacnet/read-multiple`	commission	RPM read
POST	`/api/bacnet/write`	write	Supervisory write (gated)
POST	`/api/bacnet/import-to-model`	integrator	Inventory → BRICK
POST	`/api/bacnet/driver/sync-discovery`	commission	Sync driver tree
POST	`/api/bacnet/poll/once`	commission	Trigger poll cycle
GET	`/api/bacnet/poll/status`	read	Poll worker status
POST	`/ingest/bacnet`	read	Ingest samples into feather

Bind address: `BACNET_BIND` in commission env — see BACnet network setup. Operator guide: Discover and read.

PATCH | `/api/bacnet/driver/point` | commission | Enable poll + interval on point |
PATCH | `/api/bacnet/driver/device` | commission | Enable poll for all points on device |
PATCH | `/api/bacnet/driver/device/remap` | commission | Change instance and/or address |
DELETE | `/api/bacnet/driver/point/{point_id}` | commission | Remove point from registry |
DELETE | `/api/bacnet/driver/device/{device_instance}` | commission | Remove device |
DELETE | `/api/bacnet/driver/registry` | commission | Clear CSVs + model sync |

98.7 Faults (check-engine)

Method	Path	Description
GET	`/api/faults/catalog`	Fixed fault codes
GET	`/api/faults/status`	GREEN/YELLOW/RED aggregate
GET	`/api/faults/tree`	Fault tree by equipment
GET	`/api/faults/code/{code}`	Code metadata

98.8 Timeseries & sites

Method	Path	Description
GET	`/api/timeseries/sites`	Feather sites
GET	`/api/timeseries/series`	Series metadata
GET	`/api/timeseries/plot`	Plot payload
GET	`/api/timeseries/readings`	Raw readings
GET	`/api/sites`	Site list
GET	`/api/sites/{site_id}/frame`	Wide frame for site

98.9 Building & host

Method	Path	Description
GET	`/api/building/status`	Building summary
GET	`/api/building/alerts`	Active alerts

Method	Path	Description
GET	`/api/host/stats`	CPU/mem/disk

98.10 Local agent (Ollama)

Prefix **`/openfdd-agent`**. Role **`agent`** for tool execution.

Method	Path	Description
GET	`/openfdd-agent/context`	Composed prompt context
GET	`/openfdd-agent/tools`	Tool manifest
POST	`/openfdd-agent/tool`	Execute tool (e.g. `rules.save`)
GET	`/openfdd-agent/ollama/health`	Ollama reachability
GET	`/openfdd-agent/building-insight`	Check-engine narrative context
GET	`/openfdd-agent/zone-temps`	Zone temperature insight
POST	`/openfdd-agent/chat`	Chat completion (catalog-bound)

Optional host Ollama for building insight — not required for core FDD. Configure via compose profile or host service; see Architecture overview.

98.11 Modbus

Method	Path	Role	Description
GET	`/api/modbus/driver/tree`	operator+	Commissioning tree
GET	`/api/modbus/poll/status`	operator+	Poll worker status
POST	`/api/modbus/poll/once`	integrator+	Force poll cycle

Method	Path	Role	Description
POST	`/api/modbus/read_registers`	operator+	One-shot register read
POST	`/api/modbus/read_and_store`	operator+	Read + CSV + feather (`source=modbus`)
POST	`/api/modbus/refresh`	operator+	Re-read one register
PATCH	`/api/modbus/register/poll`	integrator+	Enable poll interval

See Driver framework.

98.12 JSON API (HTTP/HTTPS)

Method	Path	Role	Description
GET	`/api/json-api/env/status`	operator+	Env var configured flags (no secret values)
POST	`/api/json-api/presets/openweather`	integrator+	Register 3-point OpenWeather bundle + optional poll
GET	`/api/json-api/driver/tree`	operator+	Endpoints grouped by host
GET	`/api/json-api/poll/status`	operator+	Poll worker status
POST	`/api/json-api/poll/once`	integrator+	Force poll cycle
POST	`/api/json-api/request`	operator+	One-shot GET/POST (no store)
POST	`/api/json-api/read_and_store`	operator+	Request + CSV + feather (`source=json_api`)
POST	`/api/json-api/refresh`	operator+	Re-fetch endpoint
PATCH	`/api/json-api/endpoint/poll`	integrator+	Enable poll interval
DELETE	`/api/json-api/endpoint/{point_id}`	integrator+	Remove endpoint

Outbound auth: `auth_type` = `none` | `bearer` | `basic`; optional `verify_tls` (default true). See JSON API driver.

98.13 PyPI package (separate)

Library-only HTTP is **not** bundled. See Python package for `open_fdd.arrow_runtime` (Arrow rules offline).

98.14 Errors

JSON error bodies; stack traces hidden unless `OFDD_DEBUG_TRACEBACKS=1`. See LAN hardening.

99 Python package

100 Python package (`open-fdd`)

Install from PyPI:

```
        pip install open-fdd
# Arrow runtime + playground (default 3.0.1+)
        pip install "open-fdd[ml]"
# optional numpy/sklearn for offline graph ML experiments
```

100.1 Modules

Module	Purpose
`open_fdd.arrow_runtime`	**Default** — `apply_faults_arrow` on PyArrow Tables, cookbook masks, column maps
`open_fdd.playground`	Rule Lab lint/compile helpers for Arrow rules

Retired in 3.0.1+: `open_fdd.engine` (YAML/pandas `RuleRunner`) is **not** shipped on PyPI. Use Operator Bridge Rule Lab or import rules from `workspace/data/rules_py/`.

100.2 When to use package-only

Scenario	Use
Lint/test Arrow rules offline	`open_fdd.arrow_runtime` + `open_fdd.playground`

Scenario	Use
Graph ML experiments (Layer B)	`open-fdd[ml]` in `experiments/` (see issue #211)
Full building operator stack	**Docker** Operator Bridge

100.3 Versioning

Publish: git tag `open-fdd-vX.Y.Z` → GitHub Actions **Publish open-fdd**.

```
import open_fdd
print(open_fdd.__version__)
```

100.4 Offline Arrow example

```
import pyarrow as pa
import pyarrow.compute as pc
from open_fdd.arrow_runtime import run_arrow_rule

code = '''
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["SAT"], float(cfg["high"]))
'''
table = pa.table({"SAT": [70.0, 90.0, 88.0]})
result = run_arrow_rule(code, table, {"high": 85})
print(result.flagged_count)
```

API surface: `open_fdd/arrow_runtime` docstrings and Arrow recipes.

101 Configuration reference

102 Configuration reference

102.1 Bridge / auth

Variable	Default	Notes
`OFDD_BRIDGE_HOST`	`127.0.0.1`	`0.0.0.0` for LAN

Variable	Default	Notes
OFDD_AUTH_SECRET	—	Required for LAN
OFDD_AUTH_DISABLED	0	Localhost dev only
OFDD_INSECURE_LAN_DEV	0	Lab only with auth disabled

102.2 BACnet

Variable	Notes
BACNET_BIND	ip/mask:47808 in commission.env
OFDD_BACNET_WRITE_ENABLED	Default off — site-specific name may vary

102.3 Docker edge

Variable	Notes
OPENFDD_IMAGE_TAG	GHCR tag for Ansible deploy
openfdd_docker_pull_from_ghcr	Ansible host_var

Rule Lab Python rules use rules_store.json + workspace/data/rules_py/*.py. Legacy YAML schema (docs/config_schema.json) is archived — not used by Operator Bridge 3.0.1+.

103 Glossary

104 Glossary

Term	Definition
Operator Bridge	FastAPI service serving API + static dashboard
Rule Lab	UI and APIs for Arrow apply_faults_arrow() rules
Feather store	On-disk columnar historian under workspace/data/feather_store/
fdd_input	Brick tag linking a point to a rule binding role
Check-engine	Building-level GREEN/YELLOW/RED fault summary

Term	Definition
Commission	BACnet discovery/read (and optional write) agent container
Poll driver	Scheduled BACnet RPM reads into historian
GHCR	GitHub Container Registry — `ghcr.io/bbartling/openfdd-*`

105 Appendix

106 Appendix

Reference material for integrators and library users.

Page	Content
API routes	Operator Bridge REST/WebSocket
Python package	PyPI `open-fdd`
Configuration reference	Key env variables
Glossary	Terms

107 Edge Docker deploy

108 Edge Docker deploy

Open-FDD edge hosts run **three** GHCR containers (`bridge`, `commission`, `mcp-rag`) with `workspace/` bind-mounted for site state.

Task	Doc
First deploy	Quick Start — Run with Docker images
Backup + image upgrade	Quick Start — Updating the stack
Container architecture	Architecture — Containers
BACnet polling	BACnet — Polling

Typical host layout:

```
~/open-fdd/
  docker-compose.yml    # from docker/compose.edge.yml
  workspace/            # historian, BACnet, auth — backup before
upgrades
```

Compose template: `docker/compose.edge.yml` in the repo.

109 Arrow-native runtime

110 Arrow-native runtime

Open-FDD 3.0 executes Rule Lab rules on **PyArrow tables** end to end.

```
feather_store.read_site_table()
  → fdd_runner / playground.run_arrow_table()
  → apply_faults_arrow(table, cfg, context)
  → pyarrow.compute masks
```

110.1 Authoring contract

```python
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

Cookbook helpers: `open_fdd.arrow_runtime.cookbook` (flatline, spread, OOB, after-hours fan).

Script-mode analytics rules receive `table` (PyArrow) and `cfg` in the sandbox — not pandas DataFrames.

110.2 Package layout

Module	Role
`open_fdd.arrow_runtime.backend`	Execute rule code, batch chunks
`open_fdd.arrow_runtime.cookbook`	Shared fault masks
`open_fdd.arrow_runtime.windows`	Rolling min/max, consecutive-true
`open_fdd.playground.arrow_templates`	Rule Lab starter templates